Multi-Line Insurance Policy and Claims Management System

1. Problem Statement

The organization is a mid-sized, multi-line insurance carrier operating across major domestic regions. The enterprise currently underwrites and administers three core product verticals: Vehicle (Auto), Property (Real Estate), and Life Insurance. Over the past decade, rapid expansion in policyholder acquisition and product complexity has severely strained the legacy operating model. Operations currently rely on fragmented technology stacks, manual pricing mechanisms, spreadsheet-driven rating tools, and paper/email-based claims distribution. These systemic inefficiencies have led to escalating operational expenses, slow quote turnarounds, widespread error rates in underwriting, delayed claims payouts, and severe exposure to compliance and regulatory penalties

2. Objectives

The primary objective of the Multi-Line Insurance Policy and Claims Management System is to consolidate disconnected legacy infrastructures into a single, scalable Salesforce architecture. The core operational objectives are broken down across operational, technical, and governance frameworks below.

1.1 Operational Objectives

1. Automate Real-Time Quoting: Eliminate manual calculations and offline spreadsheets by providing insurance agents with an automated screen flow that calculates multi-line premiums instantly.
2. Accelerate Claim Lifecycle: Reduce claim assignment timelines from 5–7 business days to real-time by automatically routing inbound claims directly to specialized queues based on the insurance line (Auto, Property, Life).

3. Enhance Adjuster Productivity: Deliver a reactive Lightning Web Component (LWC) dashboard that presents adjusters with visually prioritized claim tiles, active filtering, and aging indicators.

1.2 Technical & Architectural Objectives

1. Unify Data Architecture: Replace fragmented mainframes and isolated SQL databases with a single Policy__c and Claim__c object structure, utilizing specialized record types for different lines of business.
2. Implement Programmatic Rating Rules: Build an extensible, Invocable Apex rating engine (PremiumCalculator) capable of evaluating base risk, regional state surcharges (e.g., CA 15%, TX 5%), and asset risk modifiers programmatically.
3. Enforce Strict Front-End & Back-End Data Integrity: Deploy client-side and server-side validation rules (such as mandatory 17-character VIN enforcement) to prevent dirty data from entering the production pipeline.

1.3 Governance, Risk, & Compliance Objectives

1. Enforce Automated Approval Thresholds: Mandate a two-tier approval workflow (Senior Adjuster → Claims Manager) for any claim exceeding $50,000.00 before financial authorization is granted.
2. Maintain Strict Data Privacy & Regional Isolation: Configure Org-Wide Defaults (OWD) to Private and enforce criteria-based sharing rules to restrict access based on geographic territories and assigned job roles.

3. Project Challenges

1. MULTI-PRODUCT DATA POLYMORPHISM & SCHEMA BLOAT

In multi-line insurance, each product line (Auto, Property, Life) operates on fundamentally different data attributes:

- Vehicle (Auto) Insurance: Requires Vehicle Identification Numbers (VINs), model manufacture years, and state registration attributes.
- Property Insurance: Requires interior square footage metrics, construction years, structural risk factors, and real estate exposure profiles.
- Life Insurance: Requires designated beneficiary details, term durations in months, and risk profiling.

Creating a single, monolithic object without structural boundary enforcement results in massive field sprawl, high percentages of null values across records, cluttered page layouts, and frequent incomplete record saves. Conversely, splitting products into entirely separate custom objects breaks unified reporting, prevents cross-product analytics, duplicates development overhead, and destroys the single 360-degree view of the customer.

Operational & Technical Impact

- Data Quality Leakage: Agents entering Auto quotes can accidentally save records without mandatory vehicle attributes because the global object permits property fields.
- Layout Inflation: Adjusters are forced to navigate irrelevant fields when inspecting claims, slowing down assessment timelines.
- Governance Failure: Unvalidated data entry—such as malformed VINs under 17 characters—enters production databases, introducing audit and liability risks.

2. GOVERNOR LIMIT RISK & BULK RATING CALCULATION LATENCY

The premium rating algorithm requires evaluating multiple dynamic variables simultaneously:

- Base Rating Baselines: Setting standard line rates ($1,000.00 base).
- State Regulatory & Tax Surcharges: Dynamically applying regional multipliers (e.g., California's 15% rate surcharge or Texas's 5% surcharge).
- Asset-Specific Modifiers: Factoring vehicle age surcharges (vehicles manufactured prior to 2018) or structural square footage scaling.

Executing these mathematical matrices manually leads to an approximate 12% error rate in premium pricing. However, executing these rating calculations programmatically within Salesforce presents severe technical hurdles during bulk data execution (such as batch policy renewals or high-volume quoting requests).

Key Execution Risks

- SOQL Limits: Performing queries inside loop iterations during batch rating runs rapidly breaches governor limits (exceeding 100 SOQL queries).
- DML Limits: Executing database commits inside dynamic rating loops breaches the maximum limit of 150 DML statements.
- Heap Size & CPU Limits: Unoptimized memory allocation during batch operations causes transaction timeouts and unhandled Apex exceptions.

Operational & Technical Impact

- System Crashes: Improperly bulkified rating logic throws unhandled Apex runtime exceptions during batch updates, blocking agent operations.
- Financial Leakage: Misapplied state tax or asset surcharges cause mispriced policies, leading to direct revenue loss or regulatory fines.

3. UNSTRUCTURED CLAIMS INTAKE & QUEUE ROUTING BOTTLENECK

Inbound claims traditionally arrive through regional mailboxes, web forms, or call center logs and are manually reviewed before assignment.

Operational & Technical Impact

- Assignment Delays: Manual review steps introduce processing backlogs of 5 to 7 business days before an adjuster is assigned.
- Skillset Mismatch: Claims are frequently assigned without matching the adjuster's specialized domain (e.g., routing complex structural property claims to auto adjusters), leading to poor loss assessments, overpaid claims, and low customer satisfaction.
- Orphaned Claims: Lack of automated initial status tagging (Approval_Status__c = 'New') allows claims to sit in generic queues without ownership or SLA tracking.

4. HIGH-LIABILITY FINANCIAL RISK & APPROVAL GOVERNANCE (> $50,000)

High-value insurance claims represent major financial liability exposures. When claims exceeding significant cash thresholds (e.g., $50,000.00) are managed through standard pipelines without mandatory, multi-tier escalation routines, several critical risks emerge.

Operational & Technical Impact

- Unauthorized Financial Payouts: Line adjusters possess the ability to unilaterally approve high-dollar claims without senior management or risk committee sign-off.
- Record Lock Failures: Claims under review remain open to edits, allowing policy details or claim amounts to be altered while pending authorization.
- Audit Trail Failure: Lack of automated, multi-tiered approval logging (Submitted for Approval → Senior Adjuster Review → Claims

Manager Review) creates compliance violations during external regulatory audits.

5. ADJUSTER WORKSPACE FRAGMENTATION & PERFORMANCE LATENCY

Claims adjusters operating in legacy or standard tabular list views face severe usability barriers:

- Lack of Visual Prioritization: Adjusters cannot immediately distinguish high-value claims, aging claims (days open), or specific product lines without clicking into individual records.
- UI State Latency: Navigating across multiple tabs and list views causes frequent page reloads and degraded browser performance.
- Event Propagation Issues: Updating a claim's status on a dashboard often fails to dynamically refresh summary metrics across child components without forcing a full page refresh.

Operational & Technical Impact

- Decreased Adjuster Throughput: Adjusters waste up to 30% of their daily bandwidth navigating disconnected record pages to gather basic loss context.
- SLA Violations: Without real-time visual age indicators (days open tracking), aging high-priority claims sit unnoticed in adjuster queues.

6. REGIONAL SECURITY ISOLATION & COMPLIANCE BOUNDARIES

Insurance carriers operating across multiple states must comply with strict privacy regulations that prohibit regional agents in one state (e.g., Texas) from accessing policy and financial records in another state (e.g., California).

However, corporate executives and national claims managers require complete system-wide visibility.

Operational & Technical Impact

- Overly Permissive OWD Defaults: Setting default security model controls to "Public Read/Write" exposes cross-territory policy records, violating privacy laws.
- Overly Restrictive Rules: Setting default security models to "Private" without criteria-based sharing prevents regional teams from collaborating on local policies and blocks national managers from accessing territory metrics.

2. Solution Approach

1. COMPREHENSIVE ARCHITECTURAL BLUEPRINT

The solution replaces legacy mainframes and manual rating routines with an integrated, event-driven architecture built on Salesforce. The core layers and component interactions are organized as follows:

- Front-End User Layer
    - AutoQuoting_Screen_Flow: Guided screen flow providing insurance agents with an interface for multi-line quoting.
    - claimsDashboardLwc: Parent workspace component managing active filtering, dynamic state management, and real-time claim collection rendering.
    - claimTileLwc: Child component rendering visual claim details, aging indicators, and navigation actions.
- Business Logic Layer
    - PremiumCalculator: Invocable Apex rating engine executing bulk multi-variable rate calculations.
    - Claim Routing Record-Triggered Flow: Real-time queue assignment handler evaluating line types (Auto, Property, Life).

    - High-Value Claims Approval Process: Automated two-tier escalation workflow evaluating claim payout thresholds (> $50,000).
- Data & Security Layer
    - Policy__c Object: Unified policy object with Record Types (Auto, Property, Life) and Field Sets.
    - Claim__c Object: Centralized loss management object with Record Types (Accident, Property, Life).
    - Security Model: Private Org-Wide Defaults (OWD) combined with Criteria-Based Sharing Rules and Role Hierarchy access.

2. STEP-BY-STEP SOLUTION APPROACH BY MODULE

2.1 Multi-Product Data Architecture & Schema Optimization

To eliminate schema bloat and prevent field sprawl across disparate product lines without fragmenting the unified data model, the system uses Record Types, Field Sets, and declarative validation rules.

- Record Type Segregation: Segregate object behaviors using Record Types on Policy__c (Auto, Property, Life) and Claim__c (Accident, Property, Life).
- Dynamic Field Set Configuration: Group product-specific attributes into dedicated Field Sets (Auto_Policy_Fields, Property_Policy_Fields, Life_Policy_Fields) so that components and page layouts render only relevant fields.
- Database-Level Validation Rules: Enforce strict field formatting rules on specialized inputs, such as mandating a 17-character length for vehicle **identification attributes on auto policies.**

5. Milestones

Milestone 1: Core Data Model & Policy Configuration

Milestone 2: Complex Policy Issuance & Claim Routing Automation

1. **Create an Apex Class (PremiumCalculator) with a method that accepts policy data**
    - Click on setup icon, Right click on developer console, Open the link in new tab
    - At the top left corner, click on the file, then click on new, and click on Apex class, Give the name as PremiumCalculator and click on create.
    - Paste the following code in the apex class:

```
public class PremiumCalculator {

    public class PremiumInputWrapper {

        @InvocableVariable(required=true)

        public Id policyId;

    }

    @InvocableMethod(label='Calculate Policy Premium'
description='Calculates premium based on policy fields (VIN, Model
Year, State).')

    public static List<Decimal>
calculatePremium(List<PremiumInputWrapper> inputs) {

        List<Decimal> premiumResults = new List<Decimal>();
```

```
Set<Id> policyIds = new Set<Id>();



for (PremiumInputWrapper input : inputs) {
    if (input.policyId != null) {
        policyIds.add(input.policyId);
    }
List<Policy__c> policiesToRate = [
    SELECT Id, VIN__c, Model_Year__c, Policy_State__c
    FROM Policy__c
    WHERE Id IN :policyIds
];
for (Policy__c policy : policiesToRate) {
    Decimal basePremium = 1000.00;



    if (policy.Policy_State__c == 'CA') {
```

```
            basePremium *= 1.15;

        } else if (policy.Policy_State__c == 'TX') {

            basePremium *= 1.05;

        }

        if (policy.Model_Year__c != null &&
Integer.valueOf(policy.Model_Year__c) < 2018) {

            basePremium += 100;

        }

        premiumResults.add(basePremium);

    }

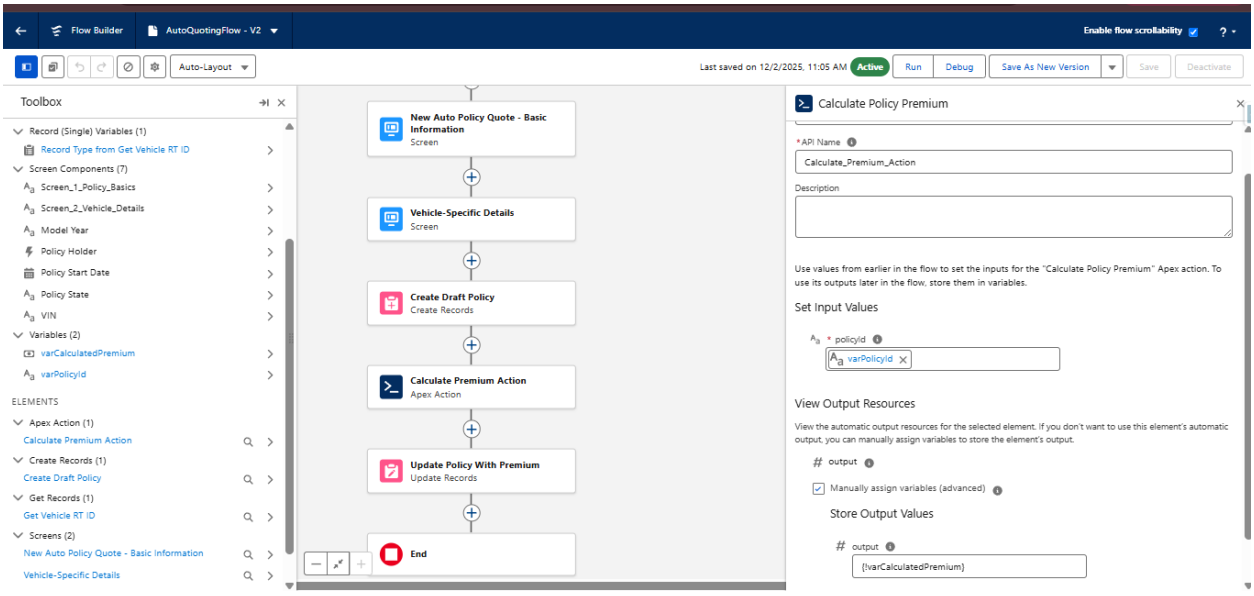
    return premiumResults;

  }

}
```

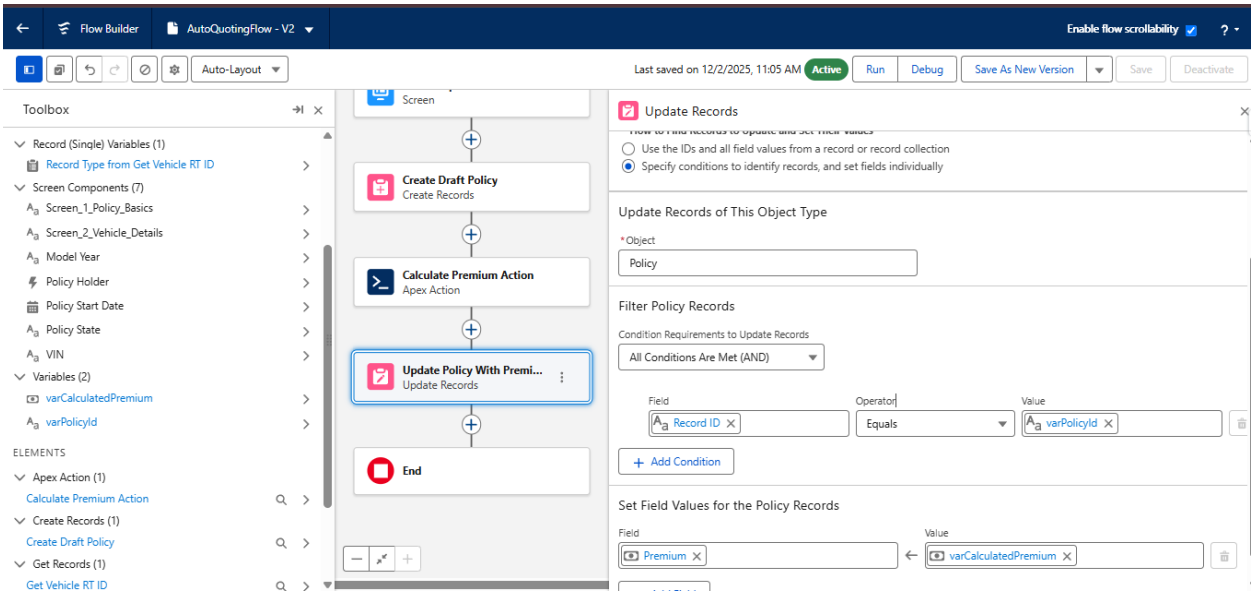
- Click on CTRL + S to save the apex class
- And this apex class need to be invoked in the AutoQuotingFlow

2. **Integrate this Apex action into the final step of the *AutoQuotingFlow* to update the draft Policy record**
    - Go to setup, in the quick find box search for flows and click on flows
    - Find for AutoQuotingFlow and open in the flow builder
    - Under the Create Records element,
    - Add an action element after the create records element.
    - Give the label as Calculate Premium Action
    - API Name will be auto populated
    - Under set Input values, For policy Id, Set the varPolicyId variable created in the previous step
    - Under view output resources, check the manually assign variables checkbox and for the output placeholder, create a resource
    - Resource type would be variable, API Name would be varCalculatedPremium, Data type would be currency, Decimal places are ‘2’ and click Done.



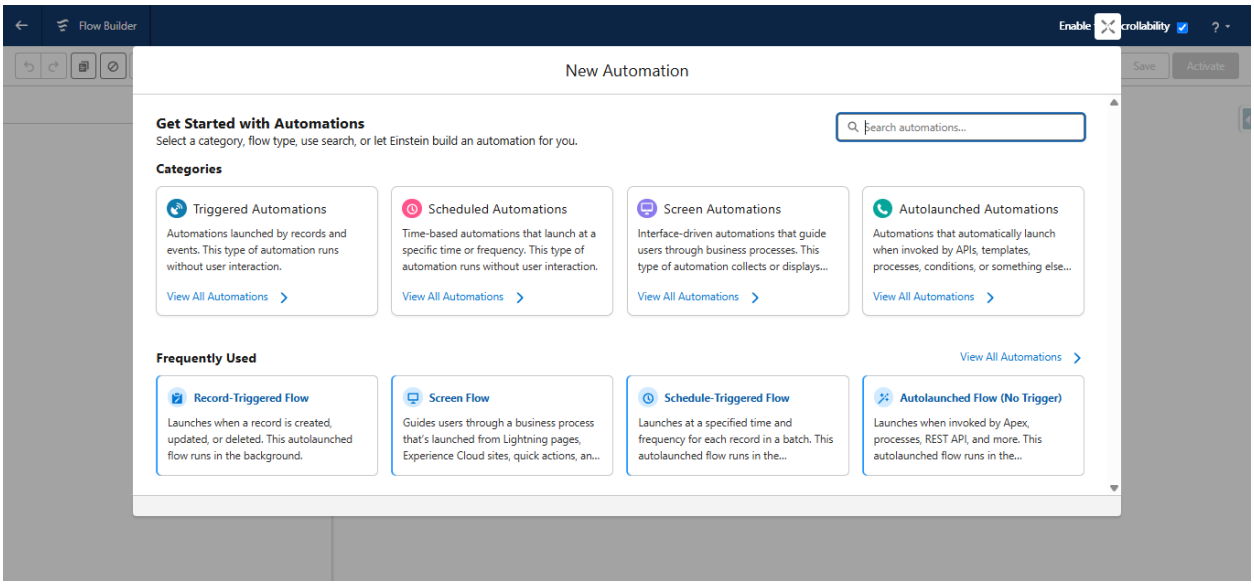
    - After the action element, add the update records element, Give the label as Update Policy With Premium

- For how to find records to update and set their values, Select Specify conditions to identify records, and set fields individually
- Select object as policy
- Select all conditions are met for filter criteria
- The conditions include:
- Record Id Equals varPolicyId variable
- Under set field values for policy record
- Assign varCalculatedPremium variable to the Premium field

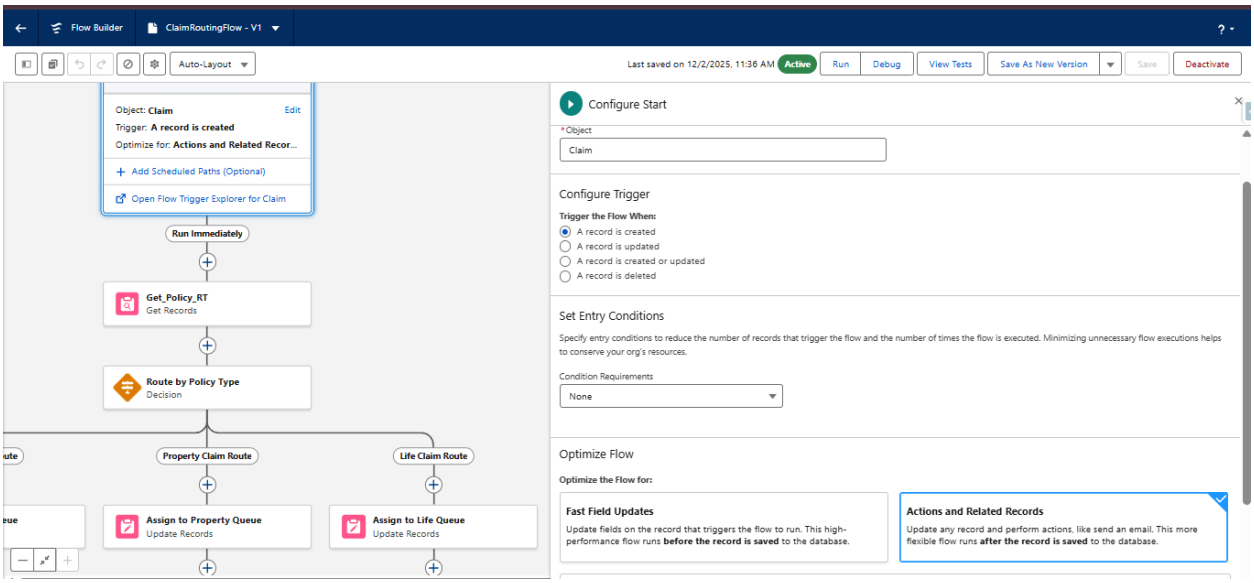


- Save the flow as a new version
- Activate the flow.

3. **Create a Record-Triggered Flow (After Save) on the Claim object.**
    - Go to setup, In the quick find box, Search for flows and click on flows
    - Click on New Flow, select Record Triggered Flow and Click on Create

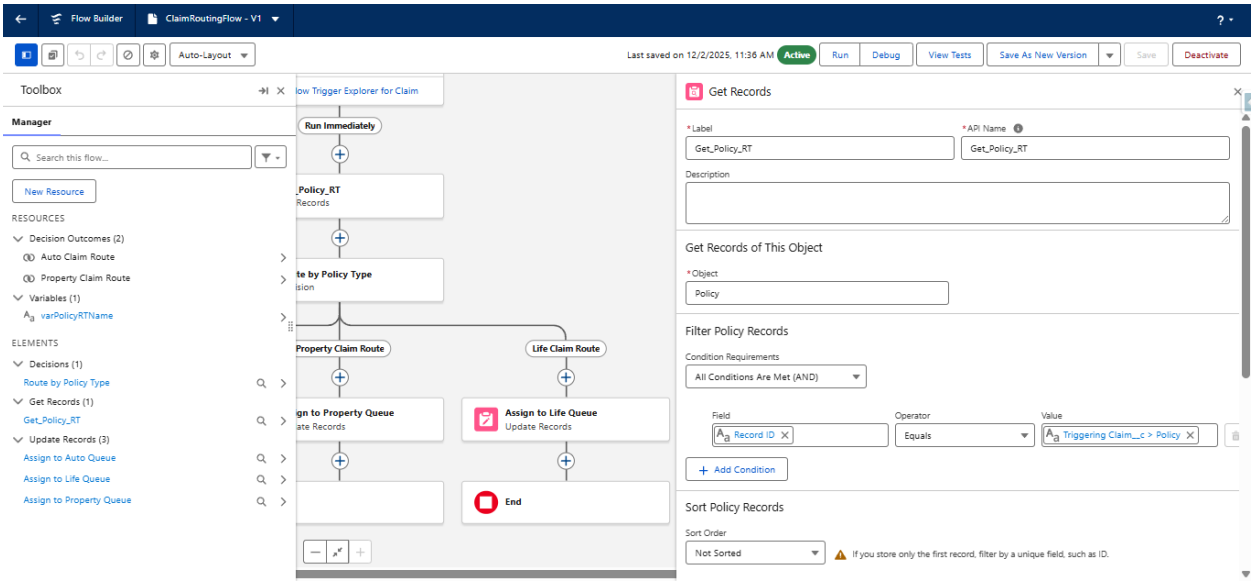


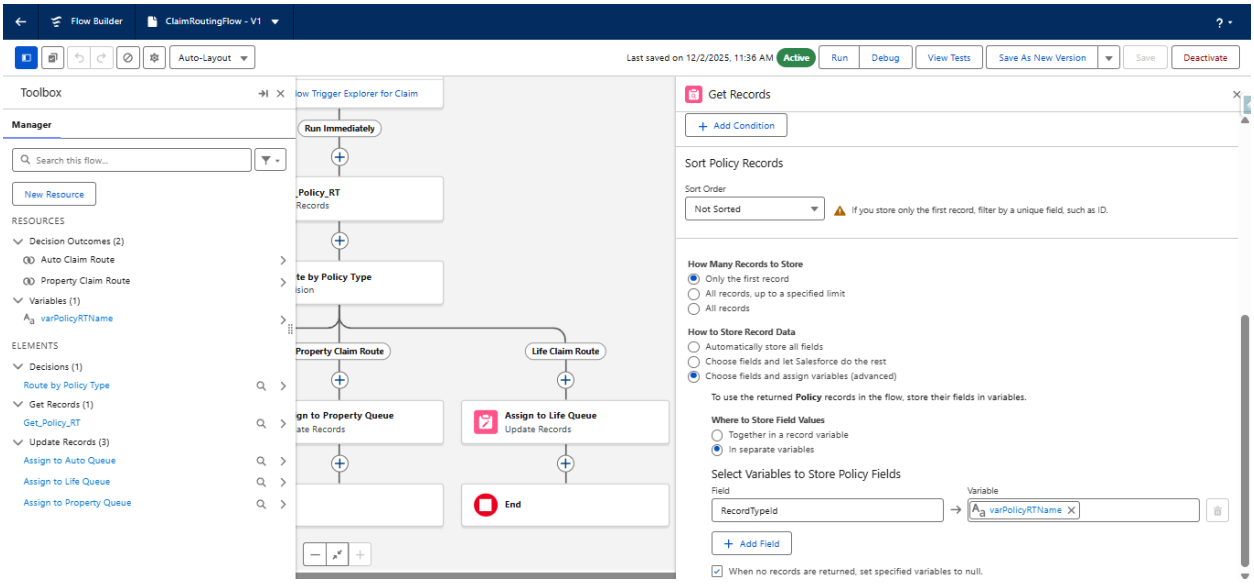
- Select Claim for an object
- Configure trigger when the record is created
- Optimize flow for action and Related Records



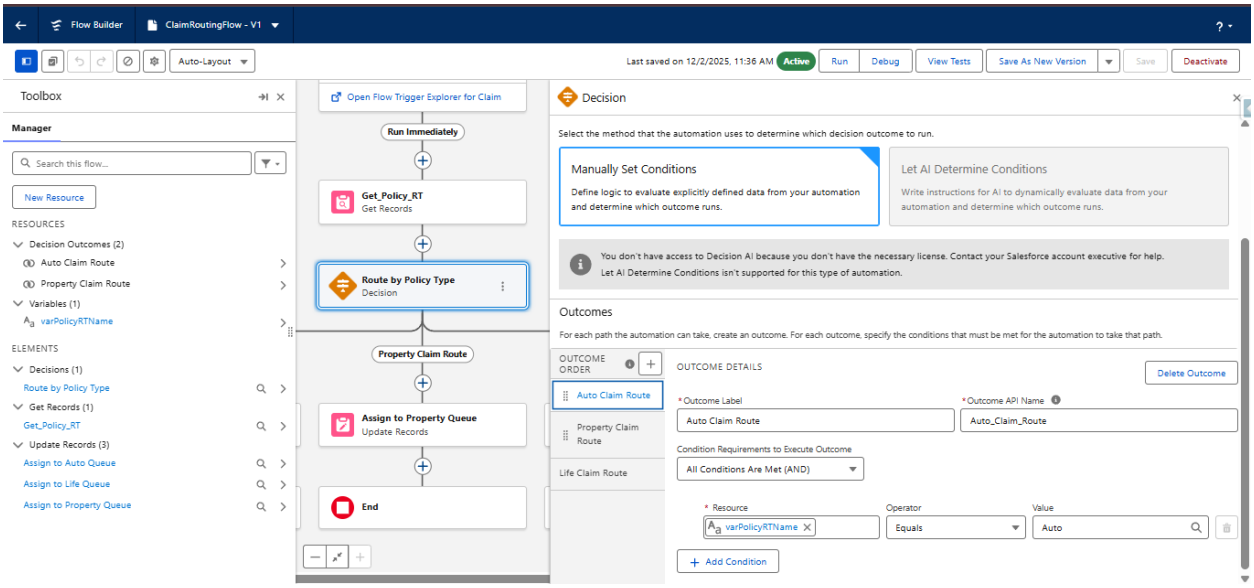
- Add Get Records element after the start and give the label as Get_Policy_RT
- Select policy for object
- Specify all conditions are met for conditional requirements

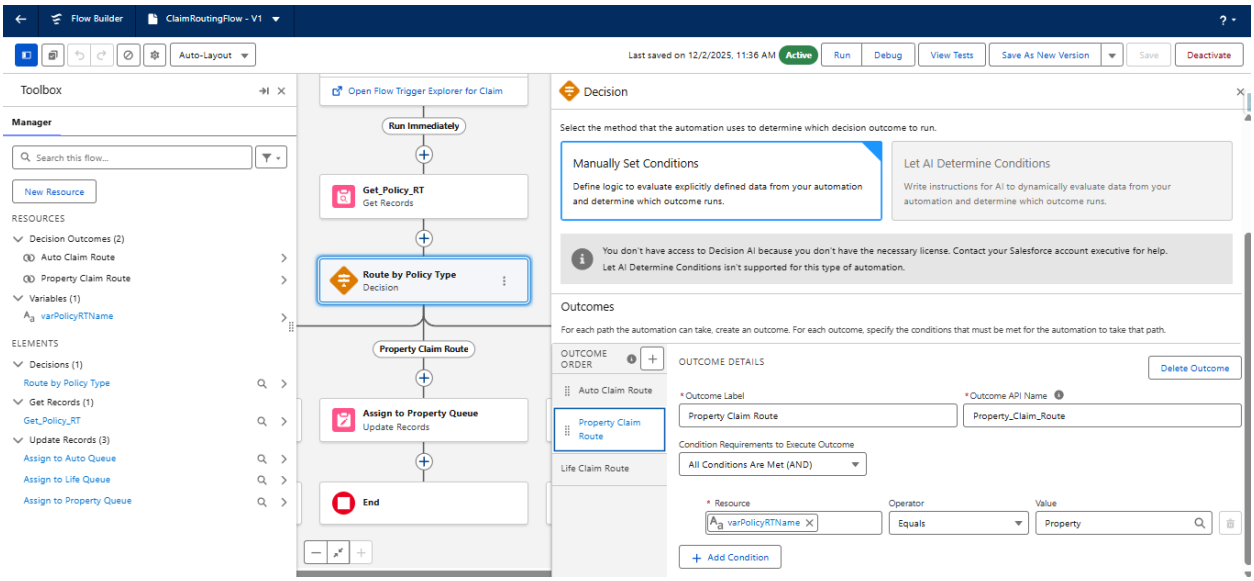
- The conditions include:
- Record Id Equals Triggering Claim → Policy
- Store only the first record
- Choose fields and assign variables for storing record data
- Select in Separate variables for where to store field values section
- Under select variables to store policy fields
- For recordTypeID field Create a resource, Resource type would be variable, API Name would be varPolicyRTName, Data type would be Text and click on Done





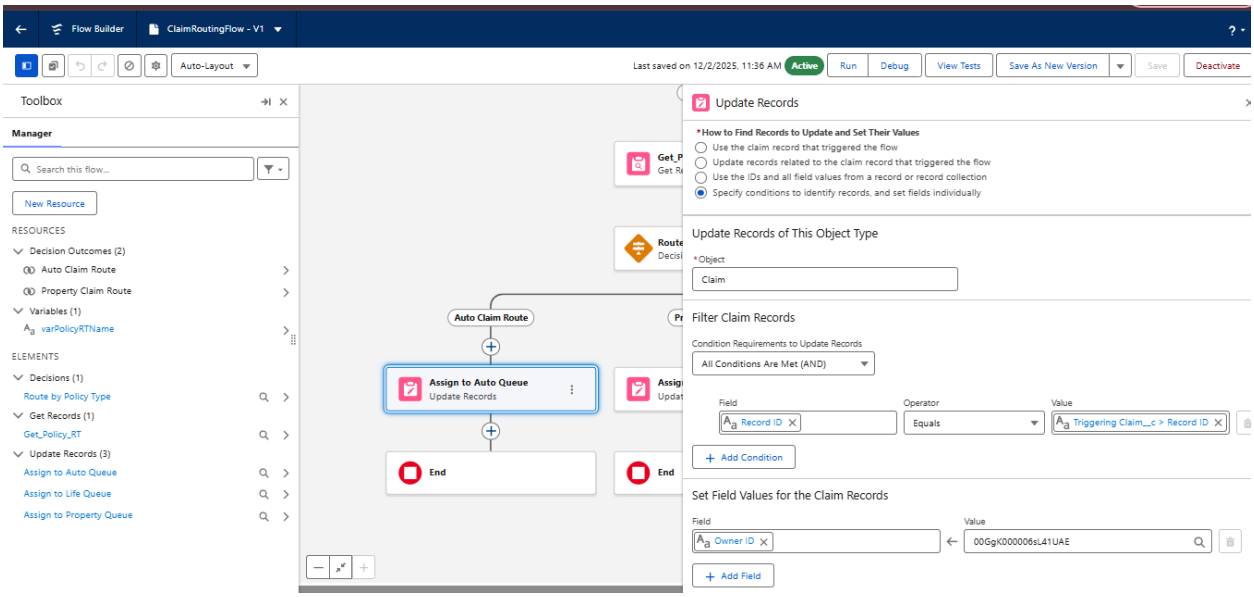
- Under Get Records Element, Add decision element
- Give the label as Route by Policy Type
- API name will be auto populated
- Select Decision Logic as Manually set conditions
- Specify the Outcomes:
- Outcome1:
    - Label as Auto Claim Route
    - API name will be auto populated
    - Select All conditions for the conditional Requirements
    - The condition includes:
    - varPolicyRTName variable equals Auto
- Outcome2:
    - Label as Property Claim Route
    - API name will be auto populated
    - Select All conditions for the conditional Requirements
    - The condition includes:
    - varPolicyRTName variable equals Property
- Default Outcome:
    - Label as Life Claim Route



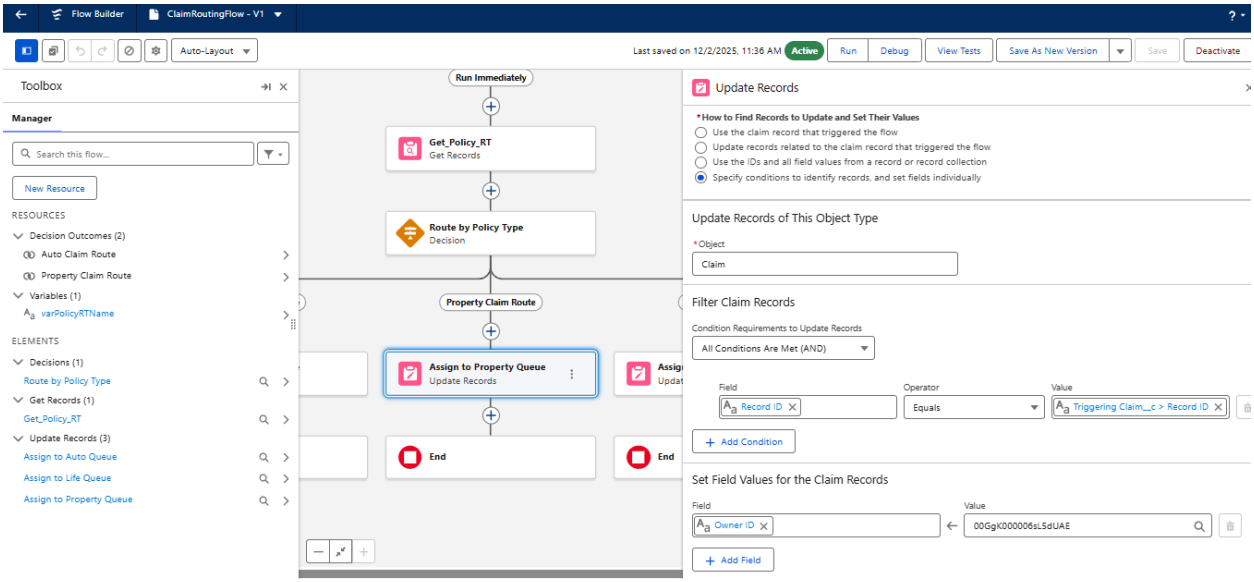


- Under Auto Claim Route Decision path, add the update records element
- Label it as Assign to Auto Queue
- API name will be auto populated
- Under How to find records to update and set their fields, select specify conditions to identify records
- Select Claim as an object
- Select All conditions are true for Conditional requirements

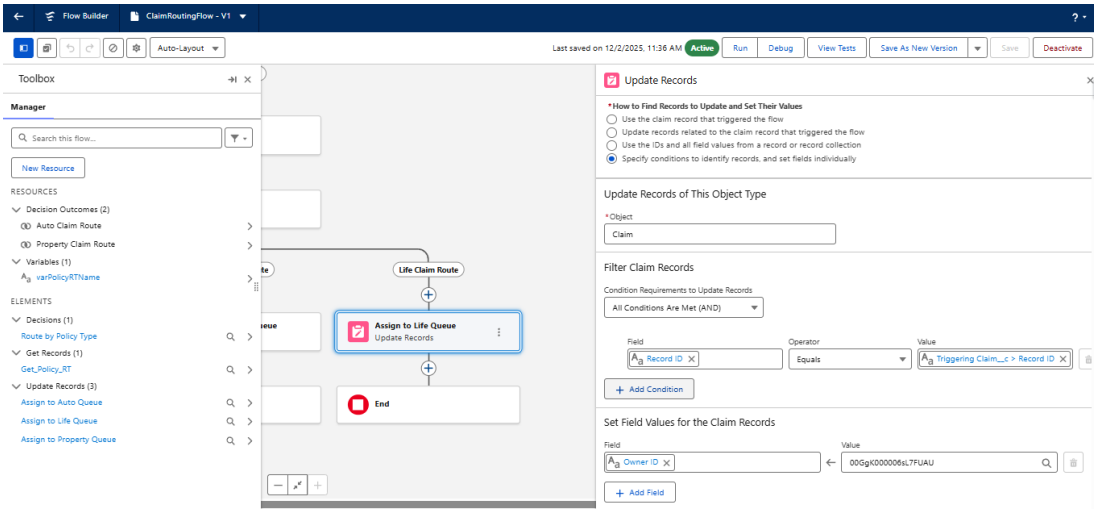
- The conditions include:
- Record Id equals Triggering Claim → Record ID
- Under Set fields for the Claim Records
- Set the hardcoded auto queue 18 digit Id for Owner ID field



- Under Property Claim Route Decision path, add the update records element
- Label it as Assign to Property Queue
- API name will be auto populated
- Under How to find records to update and set their fields, select specify conditions to identify records
- Select Claim as an object
- Select All conditions are true for Conditional requirements
- The conditions include:
- Record Id equals Triggering Claim → Record ID
- Under Set fields for the Claim Records
- Set the hardcoded Property queue 18 digit Id for Owner ID field



- Under Life Claim Route Decision path, add the update records element
- Label it as Assign to Life Queue
- API name will be auto populated
- Under How to find records to update and set their fields, select specify conditions to identify records
- Select Claim as an object
- Select All conditions are true for Conditional requirements
- The conditions include:
- Record Id equals Triggering Claim → Record ID
- Under Set fields for the Claim Records
- Set the hardcoded Life queue 18 digit Id for Owner ID field



Milestone 3: Claims Adjuster LWC Dashboard Development

1. **Create a robust Apex class (ClaimsAdjusterController) with a method using SOQL to query all Claims owned by the logged-in user, joining related Policy and Customer data. The method must return a custom Wrapper Class listing key metrics.**
    - Click on setup icon, Right click on developer console, Open the link in new tab
    - At the top left corner, click on the file, then click on new, and click on Apex class, Give the name as ClaimsAdjusterController and click on create.
    - Paste the following code in the apex class:

```
public class ClaimsAdjusterController {



    public class ClaimWrapper {
```

```
        @AuraEnabled public Id claimId;

        @AuraEnabled public String claimNumber;

        @AuraEnabled public Decimal claimAmount;

        @AuraEnabled public String status;

        @AuraEnabled public String policyType;

        @AuraEnabled public String policyHolderName;

        @AuraEnabled public Integer daysOpen;



        public ClaimWrapper(Claim__c c, Policy__c p, Contact
customerRec) {

            this.claimId = c.Id;

            this.claimNumber = c.Name;

            this.claimAmount = c.Claim_Amount__c;

            this.status = c.Approval_Status__c;
```

```
        Policy__c policyRecord = (Policy__c)p;


        if (policyRecord != null && policyRecord.RecordType != null) {
            this.policyType = policyRecord.RecordType.Name;
        } else {
            this.policyType = 'Unknown';
        }


        this.policyHolderName = (customerRec != null) ?
customerRec.Name : '';


        this.daysOpen = (c.Date_of_Loss__c != null)
                        ?
(Date.today().daysBetween(c.Date_of_Loss__c.date()))
                        : 0;
    }
```

```
    }

    @AuraEnabled(cacheable=true)
    public static List<ClaimWrapper> getAssignedClaims() {

        List<ClaimWrapper> results = new List<ClaimWrapper>();

        List<Claim__c> claims = [
        SELECT
            Id, Name, Claim_Amount__c, Approval_Status__c,
Date_of_Loss__c,
            Policy__r.Name,
            Policy__r.RecordType.DeveloperName,
            Policy__r.Customer__r.FirstName,
            Policy__r.Customer__r.LastName
```

```
        FROM Claim__c
        WHERE OwnerId = :UserInfo.getUserId()
        ORDER BY Date_of_Loss__c DESC
    ];

    for (Claim__c claim : claims) {

        Policy__c policy = claim.Policy__r;

        Contact customer = (policy != null && policy.Customer__r !=
null)
                ? (Contact)policy.Customer__r
                : new Contact();

        results.add(new ClaimWrapper(claim, policy, customer));
```

```
        }


        return results;

    }

}
```

- Save the file using CTRL + S
- This apex class will be handled in the LWC component

2. **Create a component (claimsDashboardLwc) that uses the Apex method via the @wire service and implements client-side data filtering**
    - Open the new tab in chrome and search for lightning studio extension
    - Open the first link and click on add to chrome, and confirm by clicking add Extension button
    - Now open the Lightning studio extension from your salesforce org
    - On the Top left corner, you will find Create New
    - Click on Create New and Click on Lightning Web Component
    - Enter the name of the Component as claimsDashboardLwc
    - Check the isExposed checkbox
    - Under target configurations, Select Lightning app page, Home page and Record page

- Click on Deploy
- Open the HTML file and paste the following code:

```
<template>

    <lightning-card title="My Assigned Claims
Dashboard" icon-name="standard:claim">

        <div class="slds-m-around_medium">

            <lightning-combobox

                name="policyFilter"

                label="Filter by Policy Type"

                value="All"

                options={policyTypeOptions}

                onchange={handleFilterChange}

                class="slds-m-bottom_medium"

            ></lightning-combobox>
```

```
            <template if:true={visibleClaims}>

                <div class="slds-grid slds-wrap
slds-grid_pull-padded">

                    <template
for:each={visibleClaims} for:item="claim">

                        <div key={claim.claimId}
class="slds-col slds-size_1-of-1 slds-medium-
size_1-of-2 slds-large-size_1-of-3 slds-p-
horizontal_small">

                            <c-claim-tile-lwc
claim-data={claim}></c-claim-tile-lwc>

                        </div>

                    </template>

                </div>

                <p class="slds-m-top_medium slds-
text-heading_small">Total Claims Assigned:
{visibleClaims.length}</p>

            </template>
```

```
            <template if:false={visibleClaims}>

                <p>No claims currently assigned to
you.</p>

            </template>

        </div>

    </lightning-card>

</template>
```

- Open the JS file and paste the following code:

```
import { LightningElement, wire, track } from
'lwc';

// Import the Apex method

import getAssignedClaims from
'@salesforce/apex/ClaimsAdjusterController.getAssig
nedClaims';
```

```
export default class ClaimsDashboardLwc extends
LightningElement {

    // List to hold all data retrieved from Apex

    @track allClaims = [];

    // List to display after filtering

    @track visibleClaims = [];

    error;



    // Wire service to call the Apex method

    @wire(getAssignedClaims)

    wiredClaims({ error, data }) {

        if (data) {

            this.allClaims = data;

            this.visibleClaims = data; // Initially
display all claims
```

```
            this.error = undefined;

        } else if (error) {

            this.error = error;

            this.allClaims = undefined;

            this.visibleClaims = undefined;

            console.error('Error retrieving
claims:', JSON.stringify(error));

        }

    }



    // Client-side filtering logic (Required by M3)

    handleFilterChange(event) {

        const selectedPolicyType =
event.target.value;
```

```
        if (selectedPolicyType === 'All') {

            this.visibleClaims = this.allClaims;

        } else {

            // Filter claims where policyType
matches the selected value

            this.visibleClaims =
this.allClaims.filter(claim =>

                claim.policyType ===
selectedPolicyType

            );

        }

    }



    get policyTypeOptions() {

        // Options for the filter dropdown
```

```
        return [

            { label: 'All Policy Types', value:
'All' },

            { label: 'Auto', value: 'Auto' },

            { label: 'Property', value: 'Property'
},

            { label: 'Life', value: 'Life' }

        ];

    }

}
```

- Open the meta-xml file and paste the following code:

```
<?xml version="1.0"?>

<LightningComponentBundle
xmlns="http://soap.sforce.com/2006/04/metadata">

    <apiVersion>63.0</apiVersion>
```

```
    <isExposed>true</isExposed>

    <targets>

        <target>lightning__AppPage</target>

        <target>lightning__HomePage</target>

    </targets>

</LightningComponentBundle>
```

3. **Create a reusable component (claimTileLwc) to display essential information for each claim**
    - Open the Lightning studio extension from your salesforce org
    - On the Top left corner, you will find Create New
    - Click on Create New and Click on Lightning Web Component
    - Enter the name of the Component as **claimTileLwc**
    - Check the isExposed checkbox
    - Under target configurations, Select Lightning app page, Home page and Record page
    - Click on Deploy
    - Open the HTML file and paste the following code:

```
<template>
```

```
    <lightning-layout-item size="12"
padding="around-small">

        <div class="tile slds-box slds-p-
around_medium slds-theme_default">

            <header class="slds-media slds-
media_center slds-m-bottom_small">

                <div class="slds-media__figure">

                    <lightning-icon icon-
name={policyIcon} alternative-
text={claimData.policyType}
size="small"></lightning-icon>

                </div>

                <div class="slds-media__body slds-
m-left_small">

                    <h2 class="slds-text-
heading_medium slds-m-bottom_none">

                        Claim:
{claimData.claimNumber}

                    </h2>
```

```
                    <p class="slds-text-
body_small">Policy Type:
<strong>{claimData.policyType}</strong></p>

                </div>

            </header>



            <div class="slds-border_top slds-p-
top_small">

                <p>Policy Holder:
<strong>{claimData.policyHolderName}</strong></p>



                <lightning-formatted-number

                    value={claimData.claimAmount}

                    format-style="currency"

                    currency-code="USD"

                    label="Claim Amount">
```

```
                </lightning-formatted-number>



                <p>Status: {claimData.status}</p>



                <p class="slds-text-body_regular">

                    Days Open: <strong class="slds-
text-color_error">{claimData.daysOpen}</strong>

                </p>

            </div>

        </div>

    </lightning-layout-item>

</template>
```

- Open the JS File and paste the following code

```
import { LightningElement, api } from 'lwc';
```

```
export default class ClaimTileLwc extends
LightningElement {

    // @api decorator exposes this property to the
parent component

    @api claimData;



    // Helper property to determine the correct
icon based on Policy Type

    get policyIcon() {

        if (this.claimData &&
this.claimData.policyType) {

            const type =
this.claimData.policyType.toLowerCase();

            if (type === 'auto') return
'utility:truck';
```

```
            if (type === 'property') return
'utility:home';

            if (type === 'life') return
'utility:connected_apps';

        }

        return 'utility:help';

    }

}
```

- Open the meta-xml file and paste the following code

```
<?xml version="1.0"?>

<LightningComponentBundle
xmlns="http://soap.sforce.com/2006/04/metadata">

    <apiVersion>63.0</apiVersion>

    <isExposed>true</isExposed>

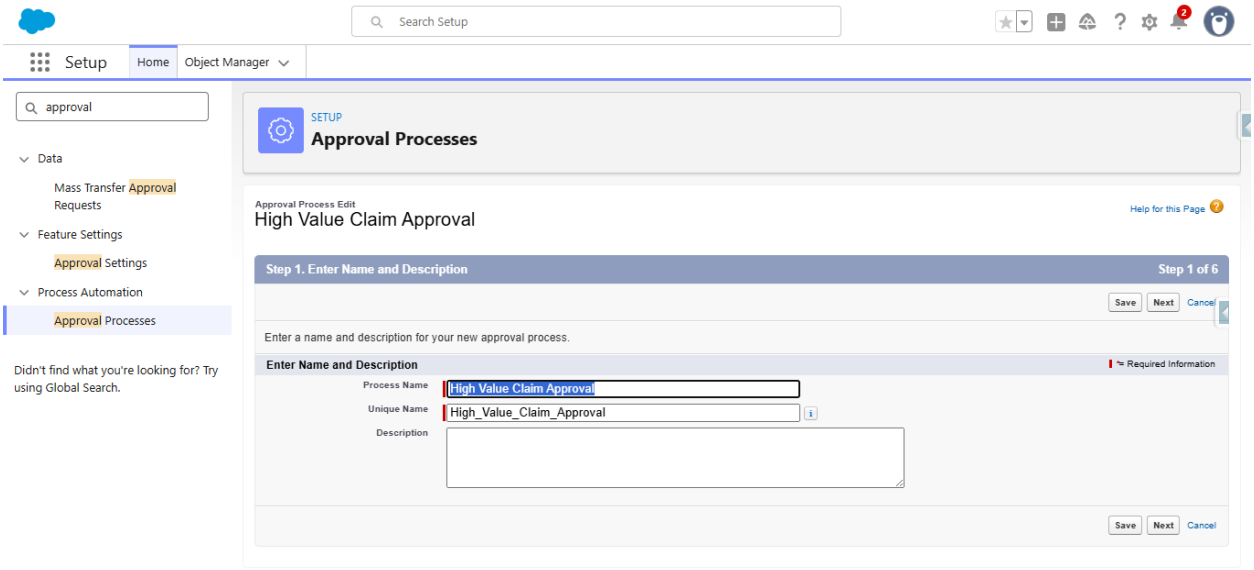
</LightningComponentBundle>
```

Milestone 4: Advanced Claim Processing, Security & Code Setup

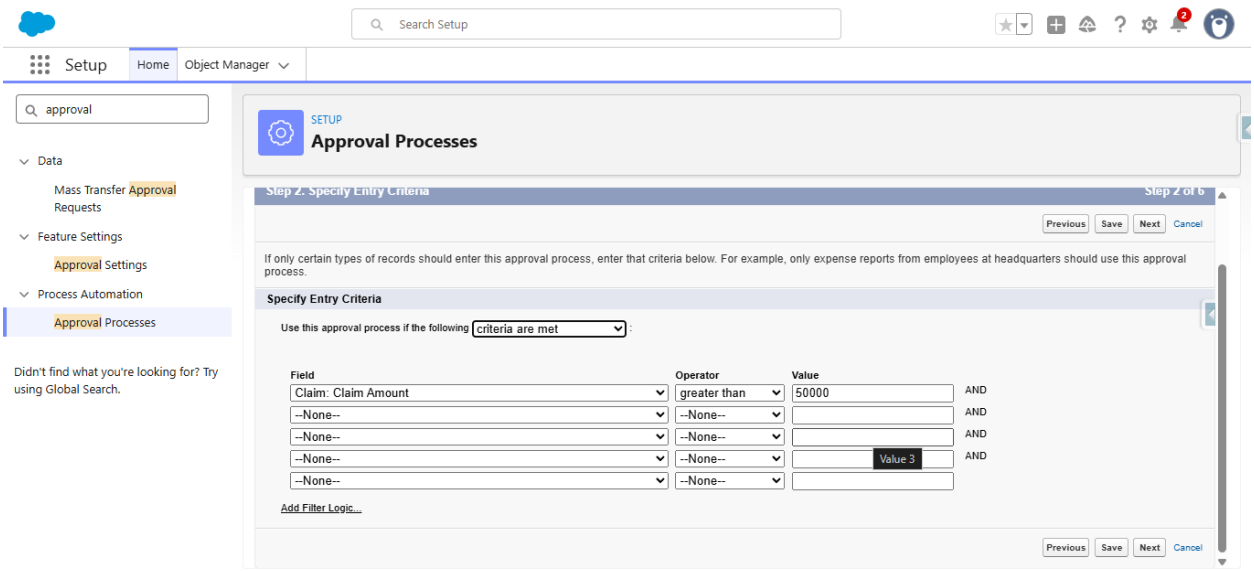
1. **Modify the Claim object to include an Approval Status field.**
    - Go to set up, Click on object manager, search for claim object and click on claim object.
    - In the left side panel, click on fields and relationships.
    - Click on new
    - Select Datatype as picklist
    - Label it as Approval Status
    - Select enter values radio button
    - Enter New, Submitted for Approval, Approved and Rejected line by line
    - Field Name will be auto populated
    - Click on Next, Next and Save.

2. **Configure the Approval Process to route first to the Senior Adjuster, then to the Department Manager, using two sequential steps.**
    - Go to setup, In the quick find box, Search for Approval process
    - Select claim object for Manage approval process for
    - Click on create new Approval Process and select use standard setup wizard
    - Give the process name as High Value Claim Approval
    - Unique Name will be auto populated

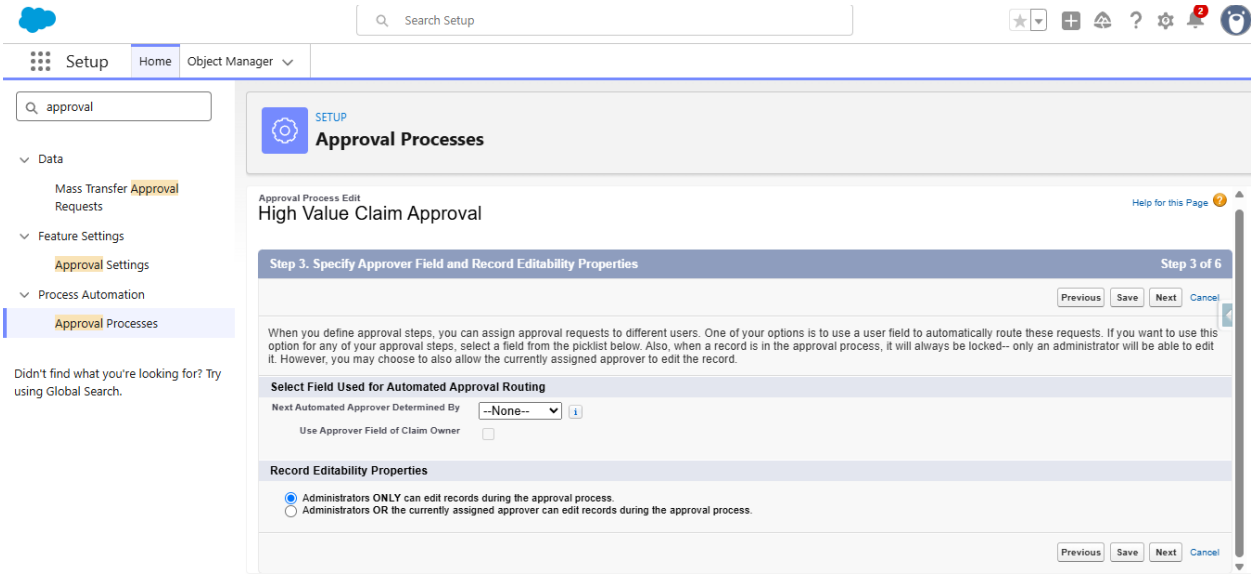
- Click on Next



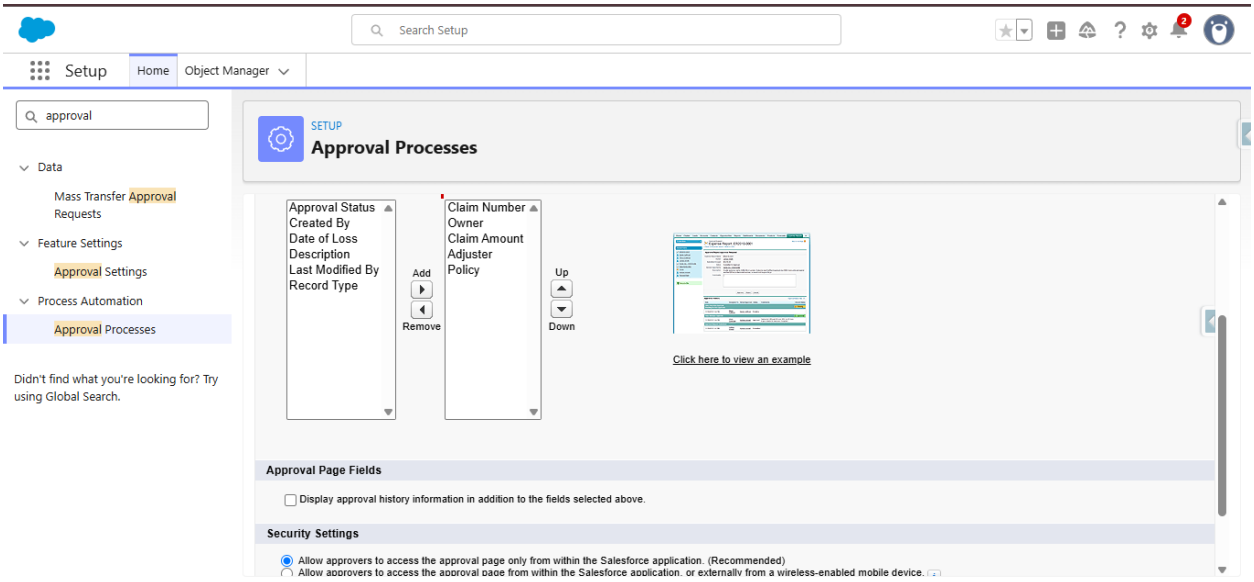
- Under specify entry criteria, Select criteria are met for Use this approval process if
- Criteria includes:
- Claim amount greater than 50000
- Click on Next



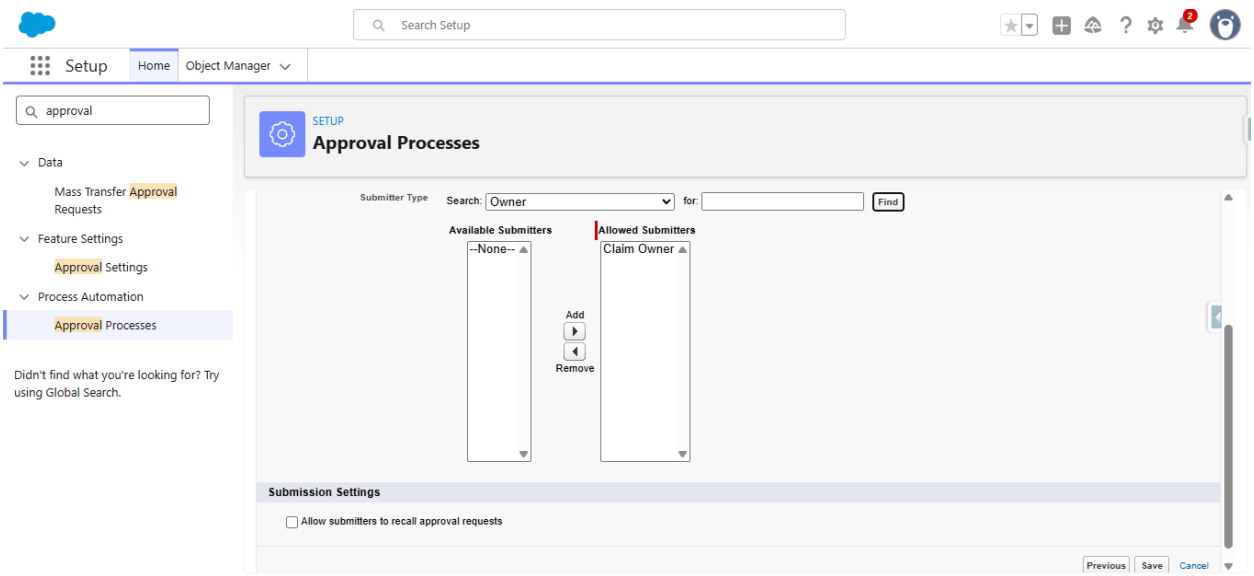
- Under Record Editability properties, Select Administrators ONLY can edit the records during the approval process
- Click on Next



- Click on next again
- Under Approval Page Layout, Select Claim Number, Owner, Claim Amount, Adjuster, Policy fields to the selected fields from available fields
- Under security settings, Select Allow Approvers to access the page only from within the salesforce application.
- Click on Next



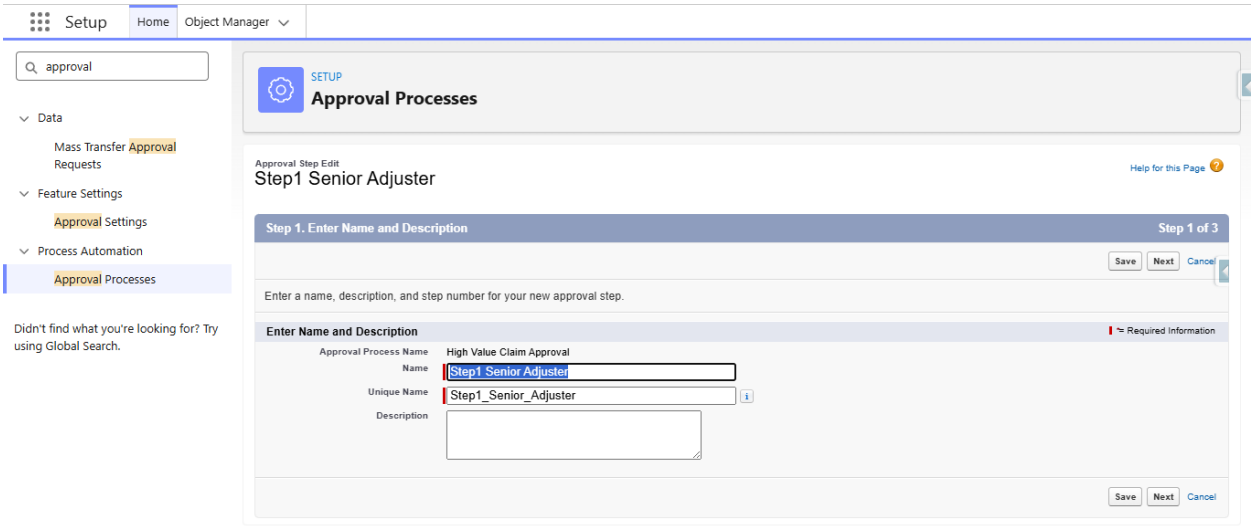
- Under Initial Submitters, Search for owner under submitter type, and select the claim owner as allowed submitters from available submitters
- Click on Save



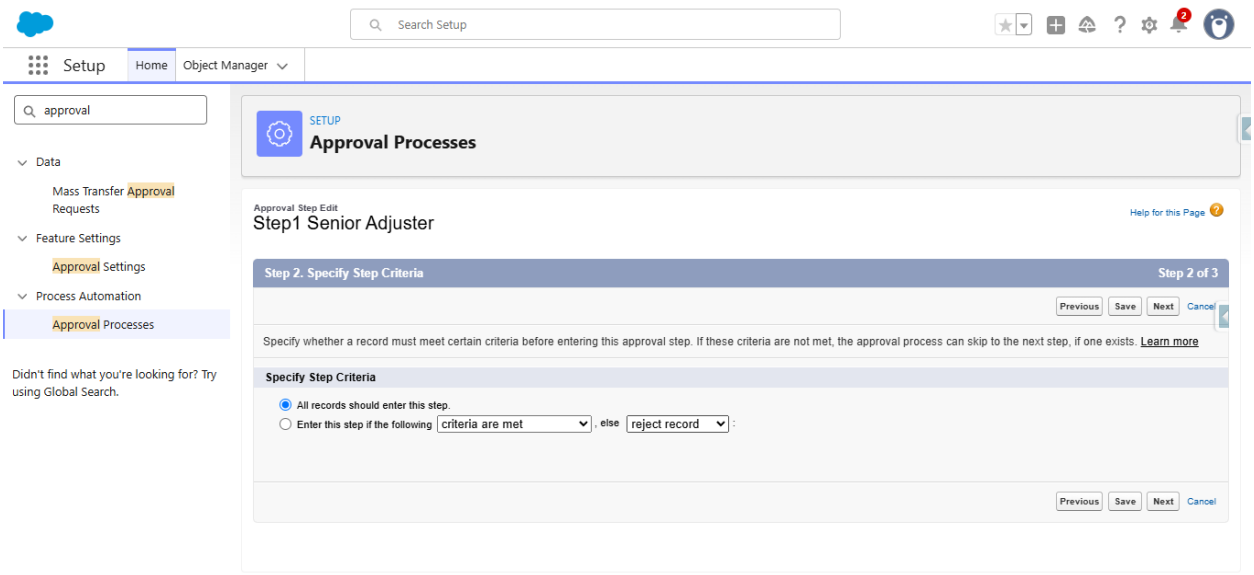
2.1 Create Approval Steps:

Step1:

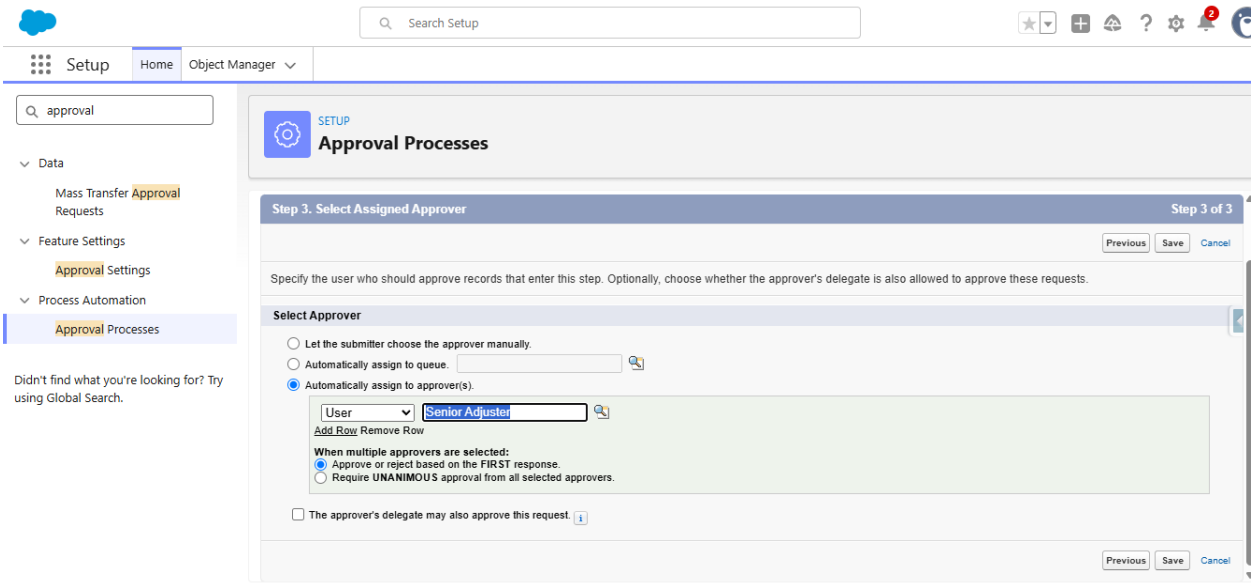
- Under initial submission actions, You can find Approval Steps
- Under approval steps, Click on New
- Give the Name as Step1 Senior Adjuster
- Unique name will be auto populated
- Click on Next



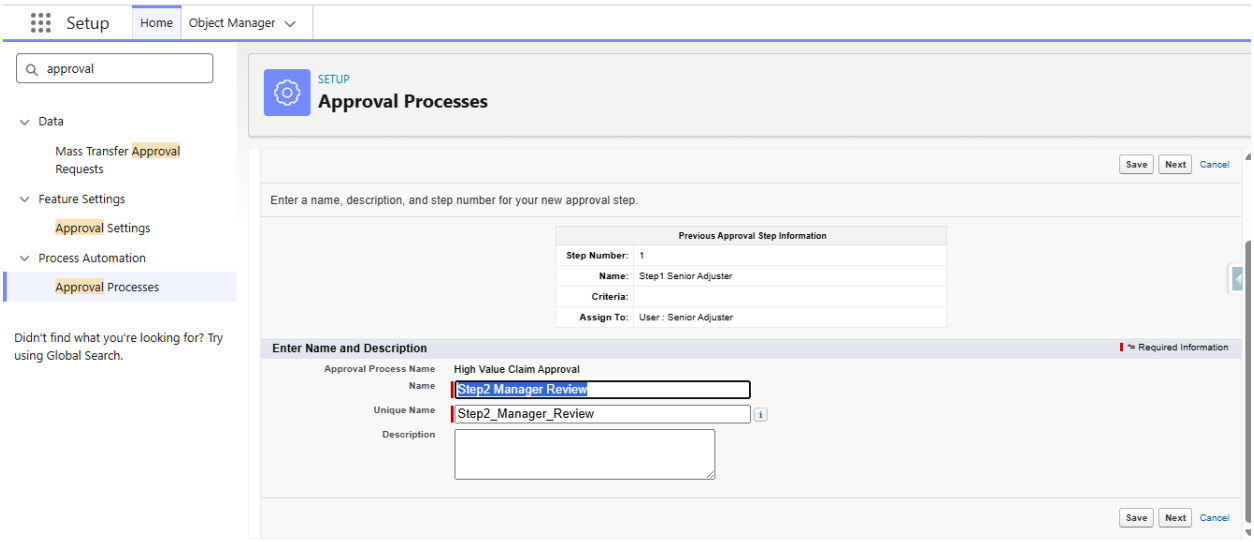
- Under specify step criteria, Select all records should enter the step
- Click on Next



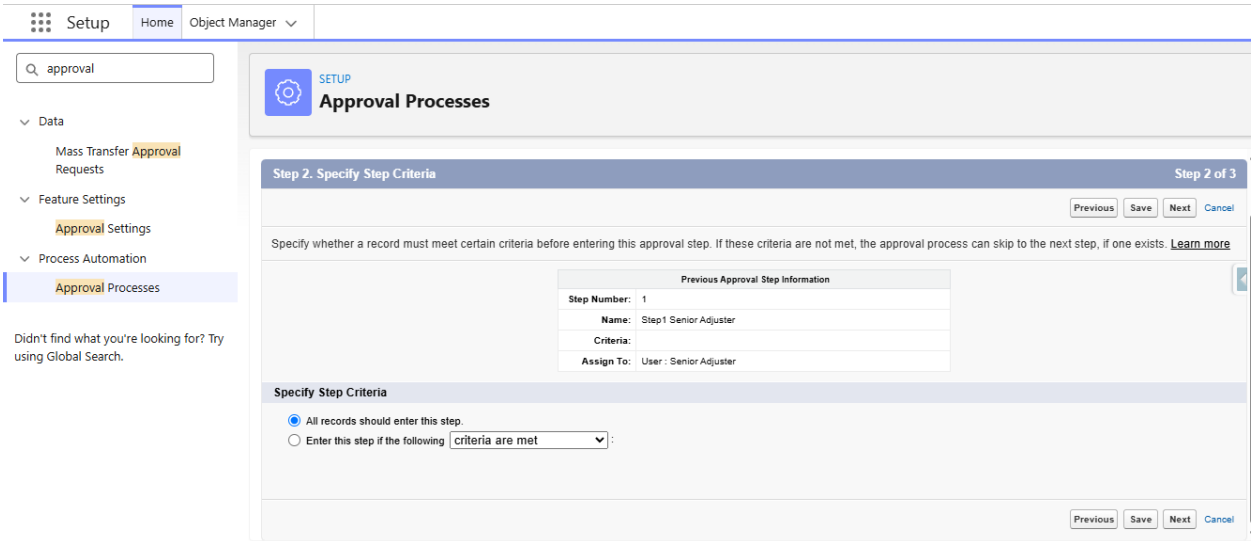
- Under select Approver, Select Automatically assign to approver
- Under that select the Senior Adjuster User
- Under when multiple approvers are selected, select Approve or Reject based on the FIRST Response
- Click on Save



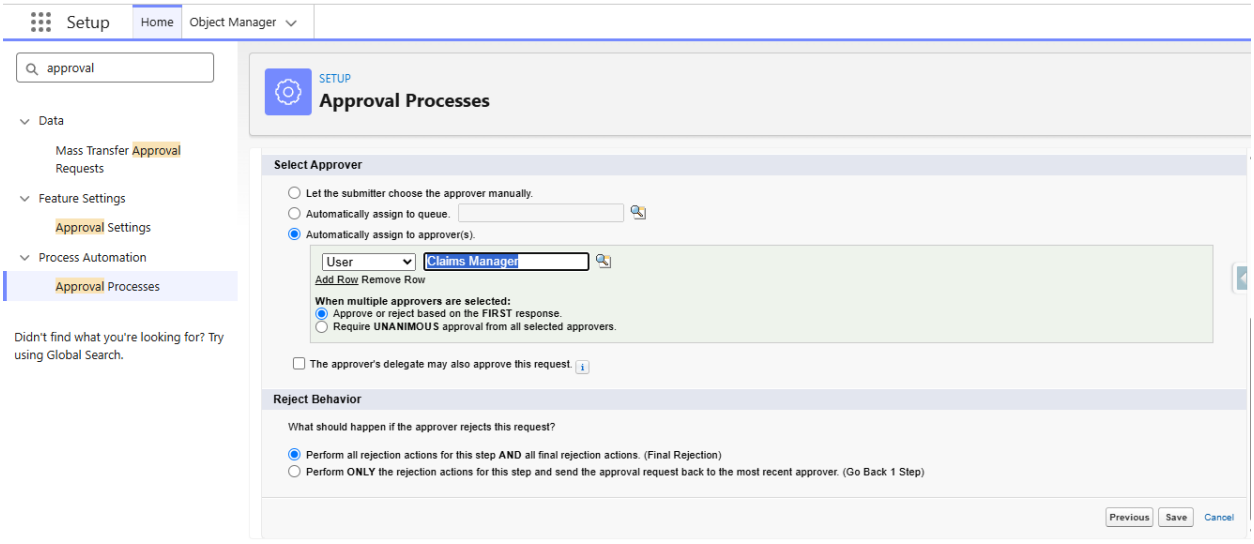
- Click on New again to create one more step
- Give the name as Step2 Manager Review
- Unique name will be auto populated
- Click on Next



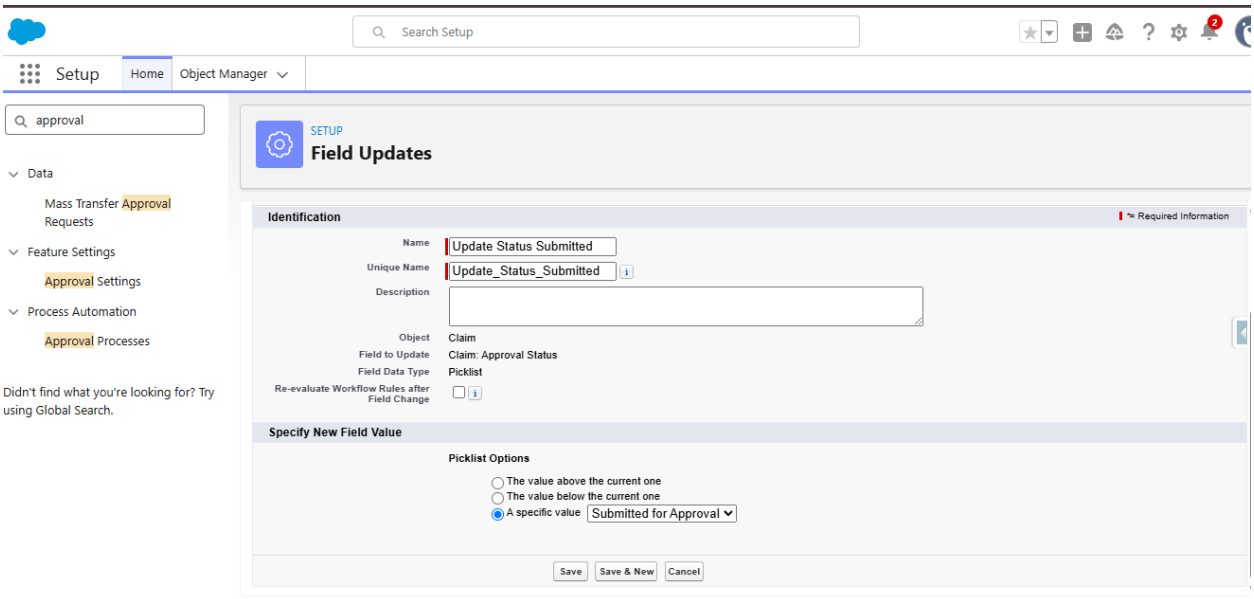
- Under specify step criteria, Select all records should enter the step
- Click on Next



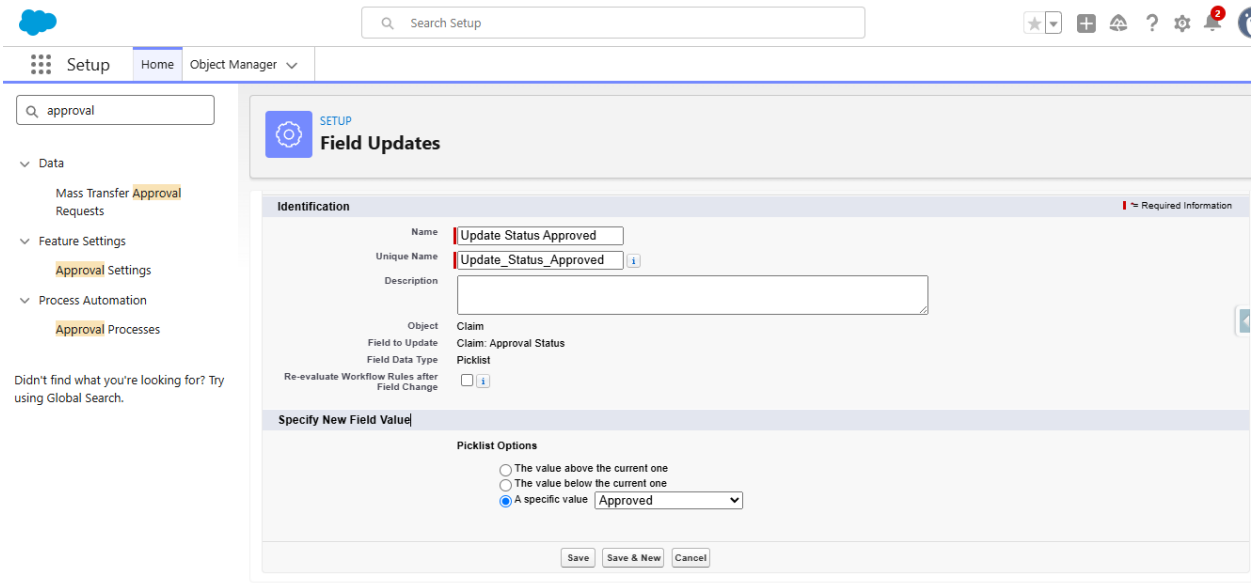
- Under select Approver, Select Automatically assign to approver
- Under that select the Department Manager User
- Under when multiple approvers are selected, select Approve or Reject based on the FIRST Response
- Under Reject behaviour, Select Perform all rejection actions for this step AND all final rejection actions
- Click on Save



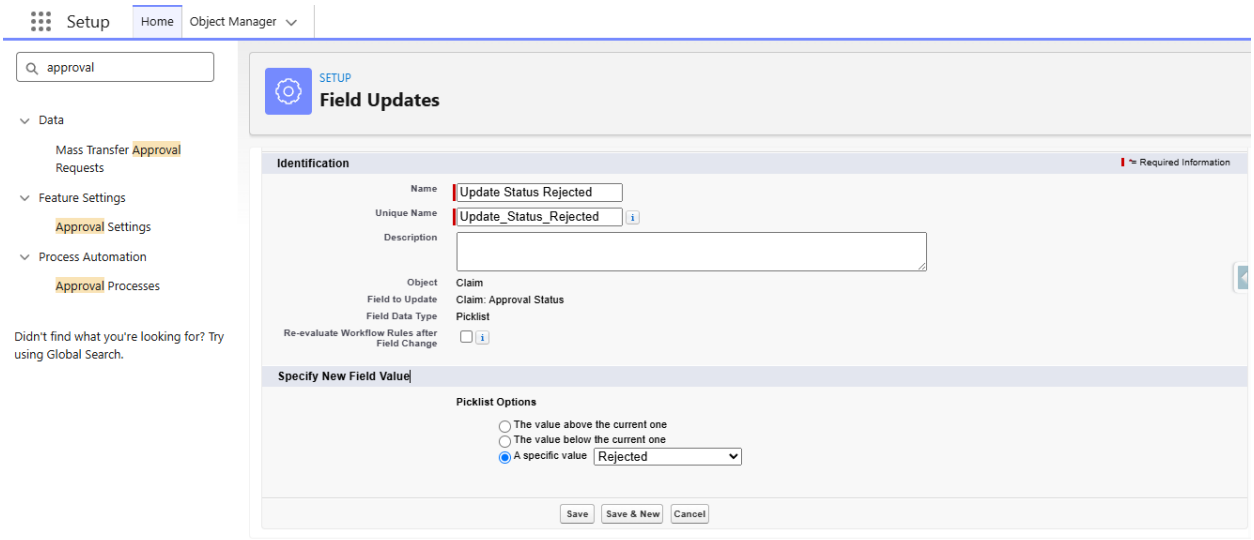
- Under Initial Submission Actions, Click on Add New and select Field Update
- Give the name as Update Status Submitted
- Unique name will be auto populated
- Select object as Claim
- Select Approval Status for Field to Update
- Select Picklist as field datatype
- Under specify new field value, Select a Specific value and select submitted for Approval
- Click on Save



- Under Final Approval Actions, Click on Add New and select Field Update
- Give the name as Update Status Approved
- Unique name will be auto populated
- Select object as Claim
- Select Approval Status for Field to Update
- Select Picklist as field datatype
- Under specify new field value, Select a Specific value and select Approved
- Click on Save



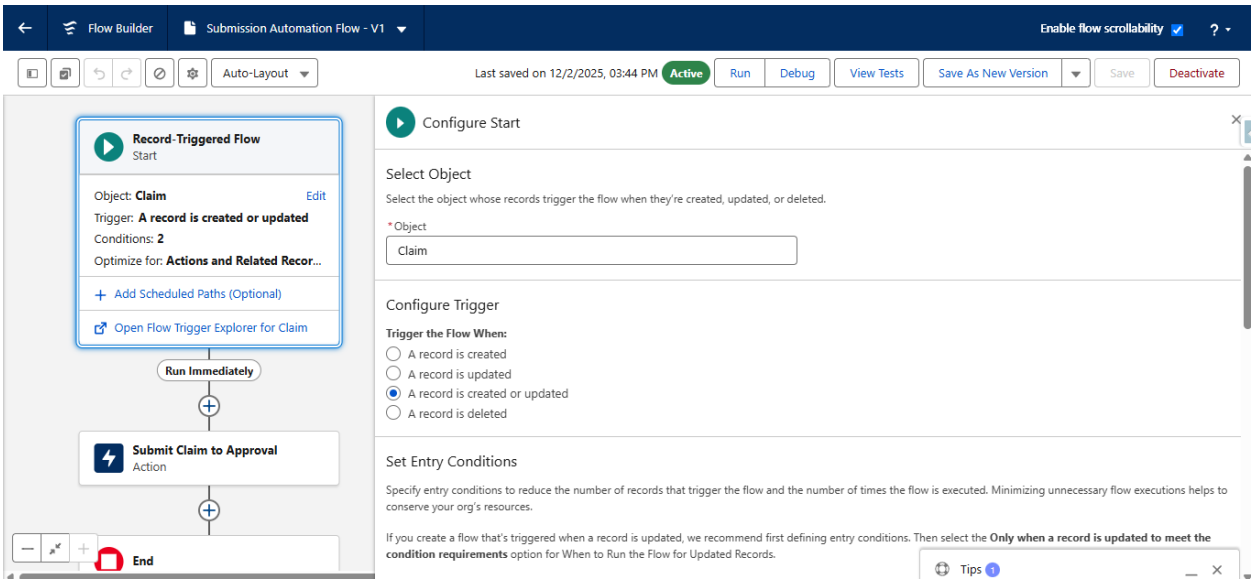
- Under Final Rejection Actions, Click on Add New and select Field Update
- Give the name as Update Status Rejected
- Unique name will be auto populated
- Select object as Claim
- Select Approval Status for Field to Update
- Select Picklist as field datatype
- Under specify new field value, Select a Specific value and select Rejected
- Click on Save

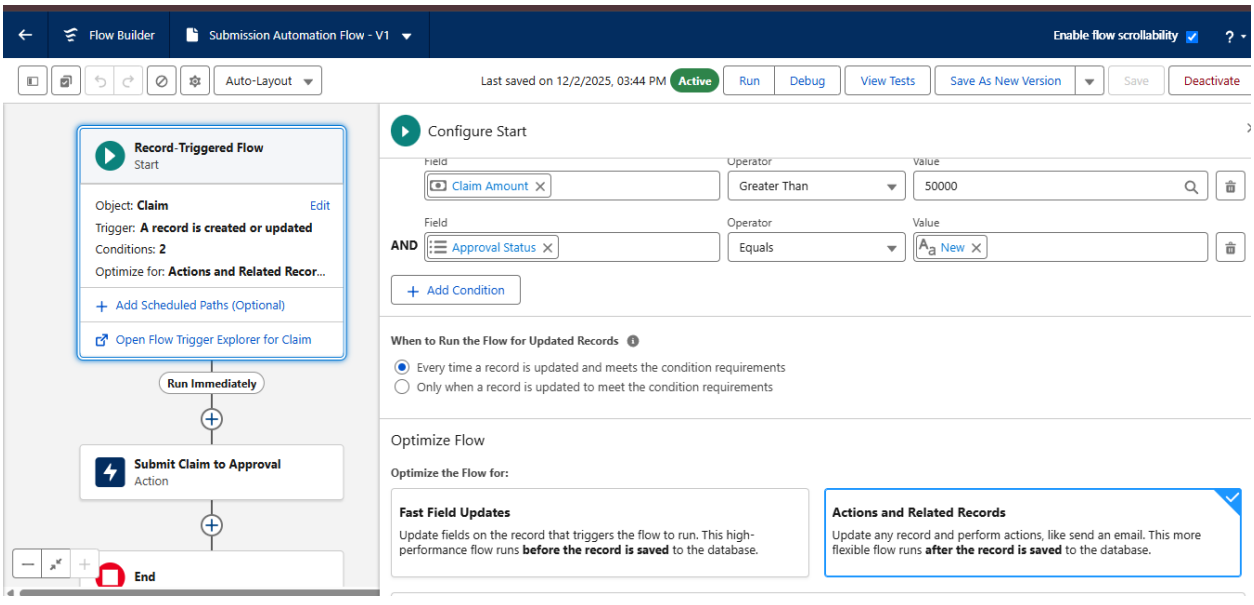


- Activate the Approval Process.

3. **Create a Record-Triggered Flow (After Save) that checks if the Claim Amount is $>\$50,000$ and submits the record to an Approval Process.**
    - Go to setup, In the quick find box, search for flows and click on flows.
    - Click on New Flow
    - Select Record Triggered Flow
    - Select object as Claim
    - Configure Trigger When a Record is created or updated
    - Select All conditions are true for conditional requirements
    - The conditions include:
        - Claim Amount Greater than 50000
        - Approval Status Equals New

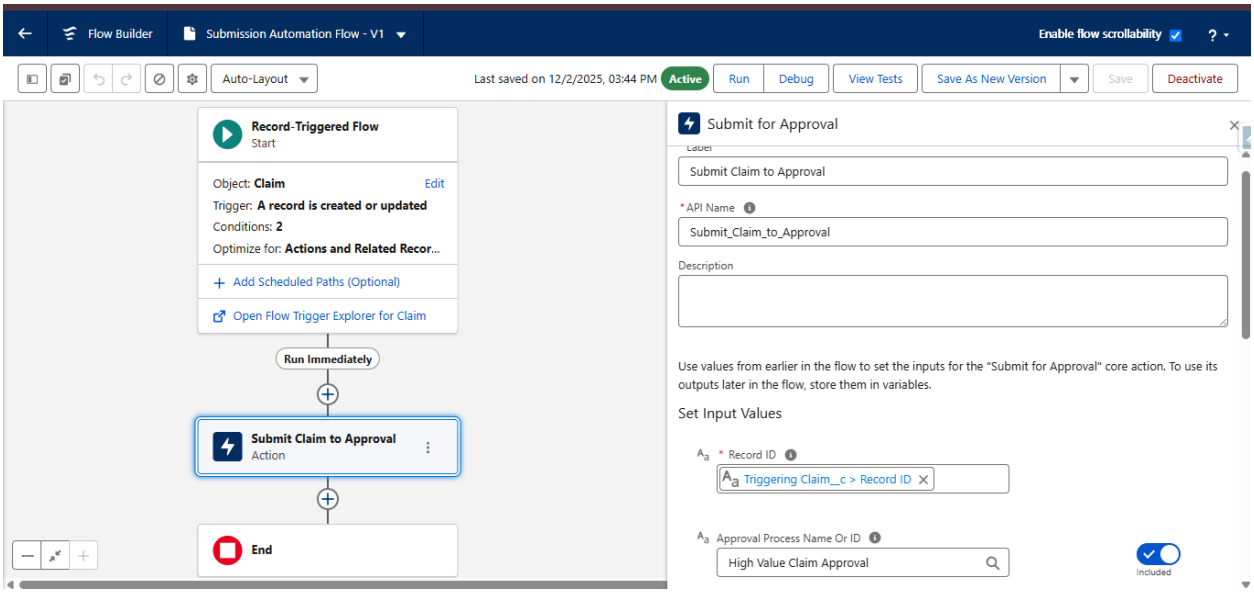
- Select Every time a record is updated and meets the conditional requirements for When to run the flow for Updated Records
- Optimize the flow for Action and Related Records





- Add an action element after the Start Element
- Label it as Submit Claim to Approval

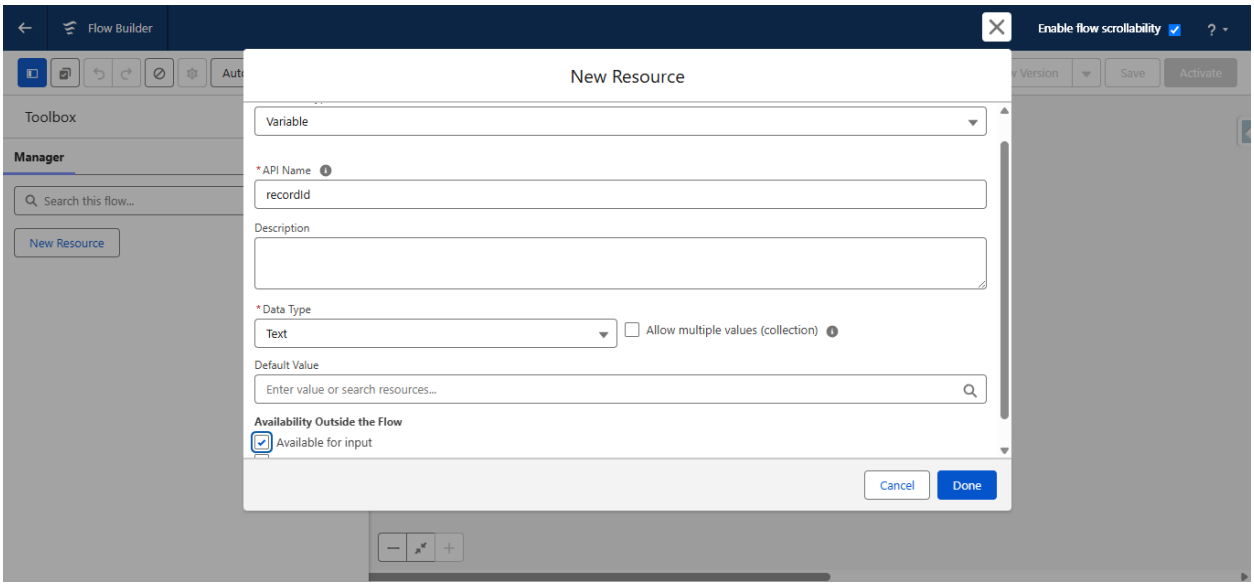
- Under Set Input Values, For Record Id, Select Triggering Claim → Record ID
- For select Approval Process Name or Id, Enter the Approval Process Name(High Value Claim Approval)



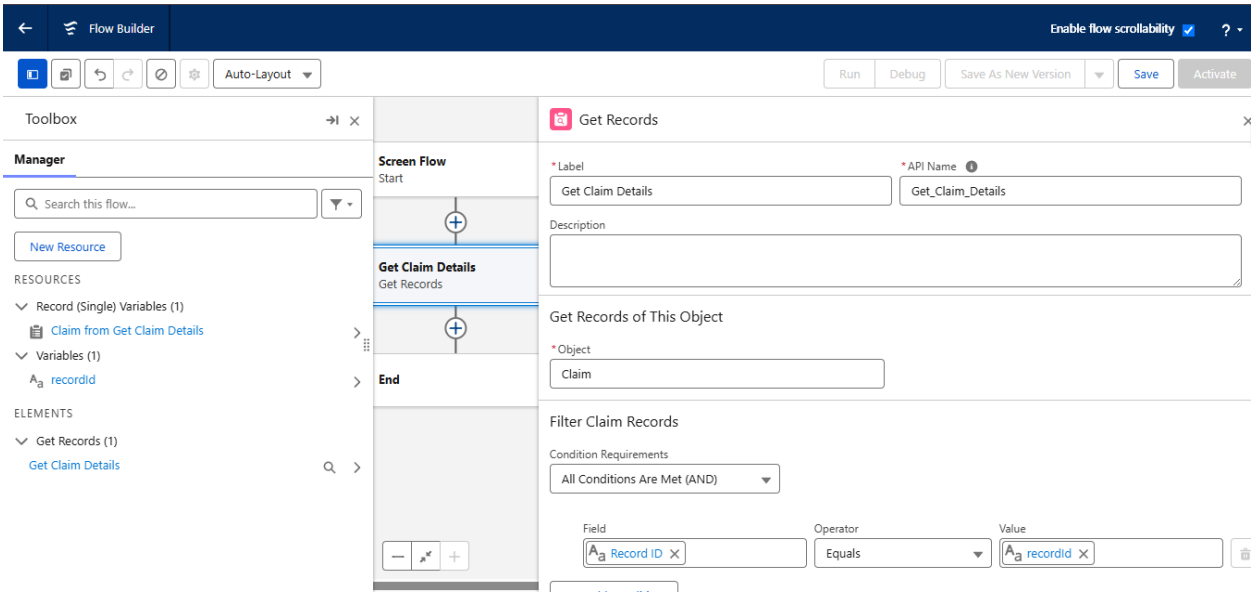
- Save the Flow with Submission Automation Flow and Activate the flow

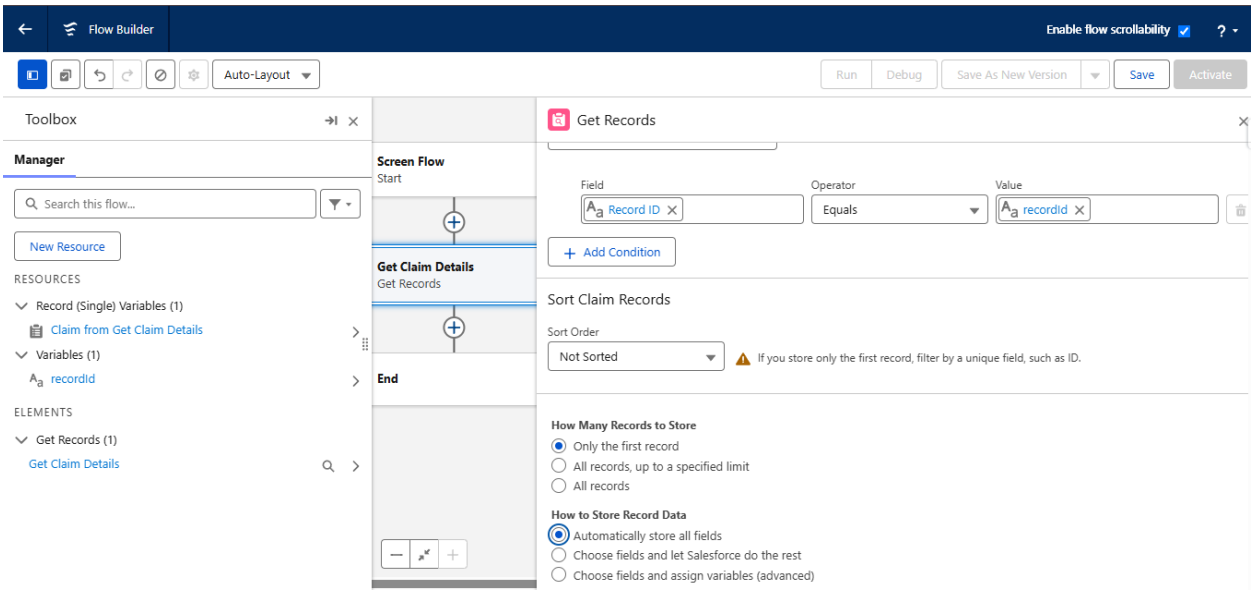
4. **Implement a Screen Flow for the approvers to quickly review details and approve/reject directly from the Claim record.**
    - Go to setup, Search for flows in the quick find box and click on flows.
    - Click on New Flow, Choose Screen Flow
    - Create a New Resource by clicking on New Resource button in the toolbox manager
    - Resource Type would be Variable

- API Name would be recordId
- Data type would be Text
- Under Availability outside the flow, Check Available for Input
- Click on Done



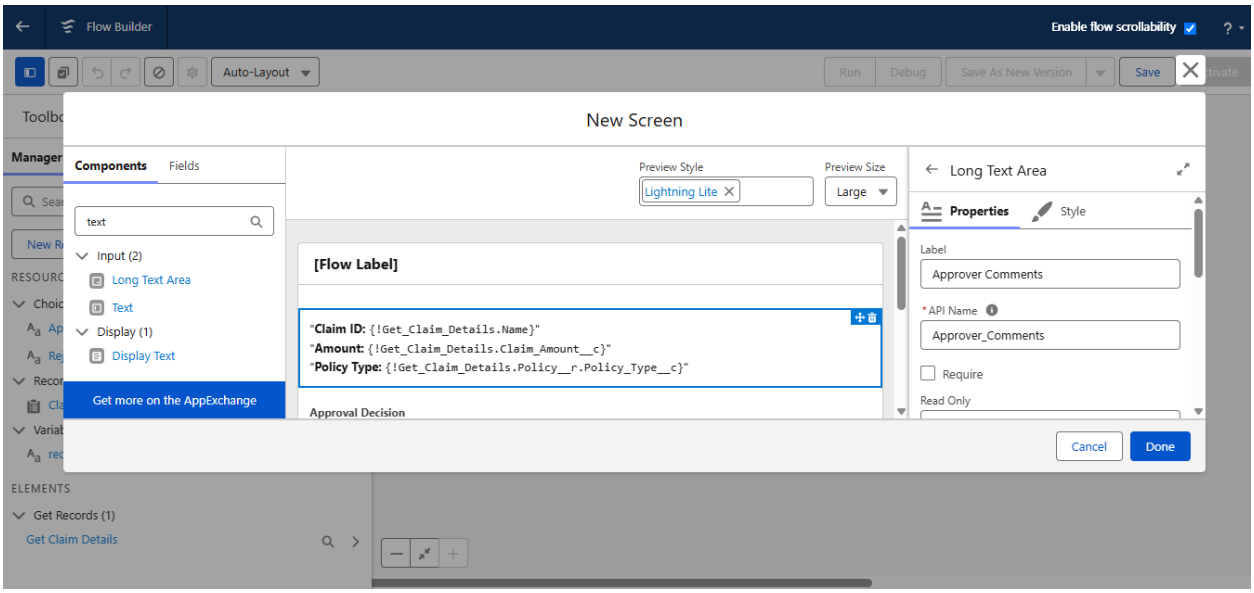
- Add the Get Records Element below the screen
- Label it as Get Claim Details
- Select object as Claim
- Select All conditions are required for conditional Requirements
- The Condition include:
- RecordID equals variable recordId
- Store only the first record and
- Automatically store all fields

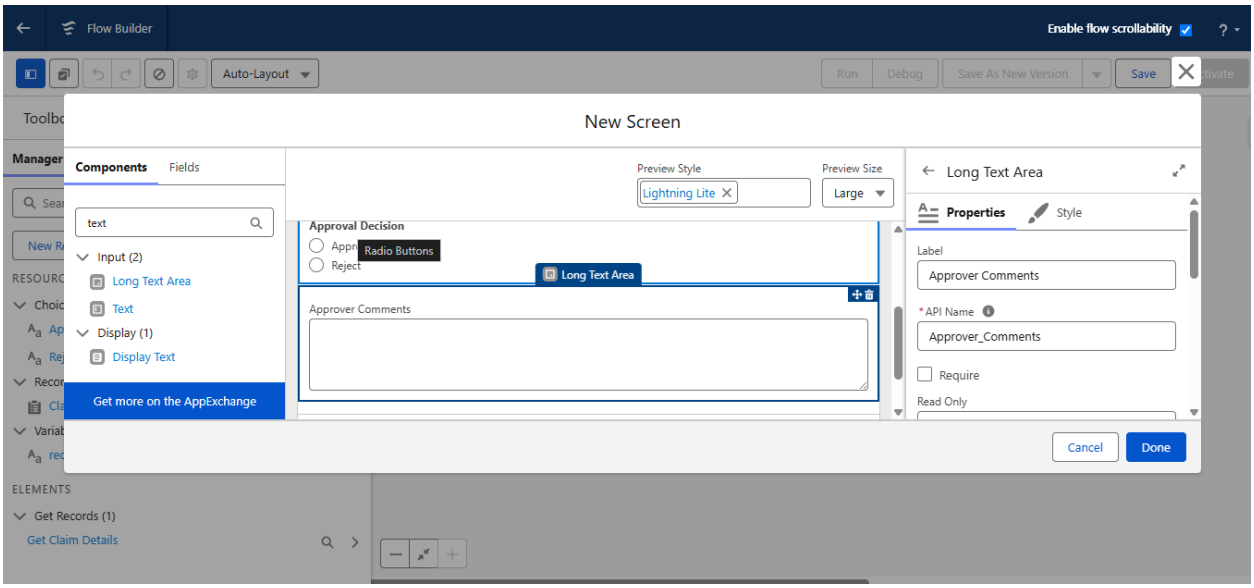




- Add a screen Element under the get Record element
- Label it as Claim Review Screen
- Add Display Text component to the flow canvas, and paste the following:
    - "**Claim ID:** `{!Get_Claim_Details.Name}`"

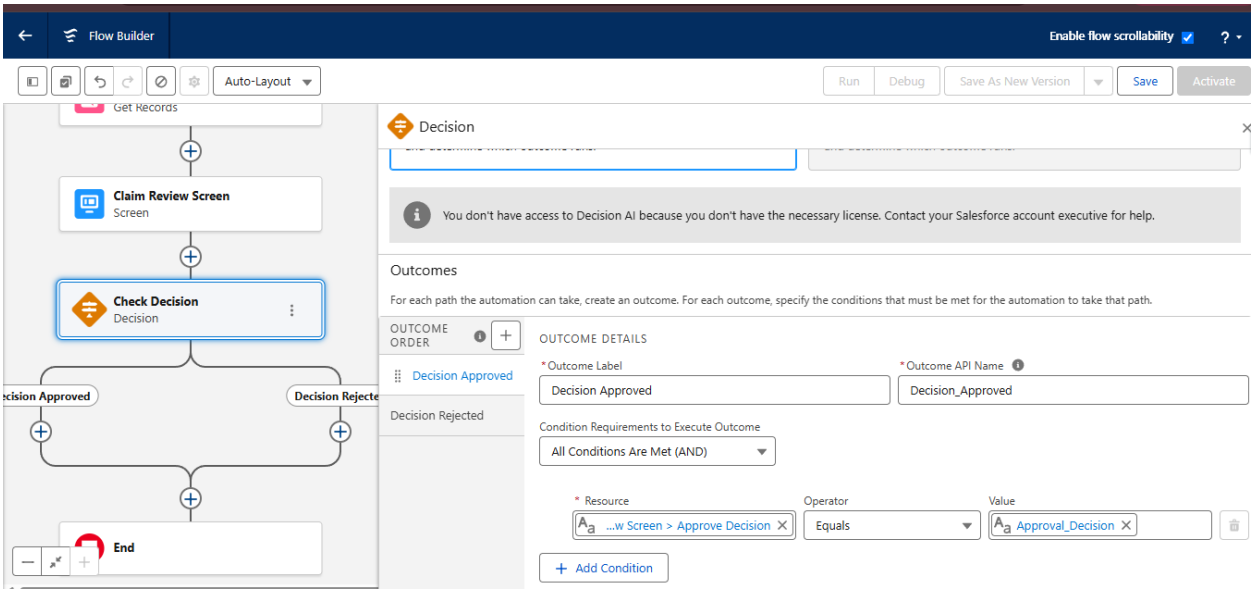
    - "**Amount:** `{!Get_Claim_Details.Claim_Amount__c}`"
    - "**Policy Type:** `{!Get_Claim_Details.Policy__r.RecordType.Name}`"
- Add the Radio buttons component, Label it as Approval Decision
- API Name would be auto populated
- For Choice, Create a resource, Resource type would be choice
- Give the choice label as Approve
- Select Datatype as Text
- Enter the choice value as Approve
- Enter the input label as Approve
- Click on Done
- Add one more Choice, Create a resource, Resource type would be choice
- Give the choice label as Reject
- Select Datatype as Text
- Enter the choice value as Reject
- Enter the input label as Reject
- Click on Done
- Add Long Text area into the flow canvas and Label it as Approver Comments
- API Name would be Auto Populated
- Click on Done



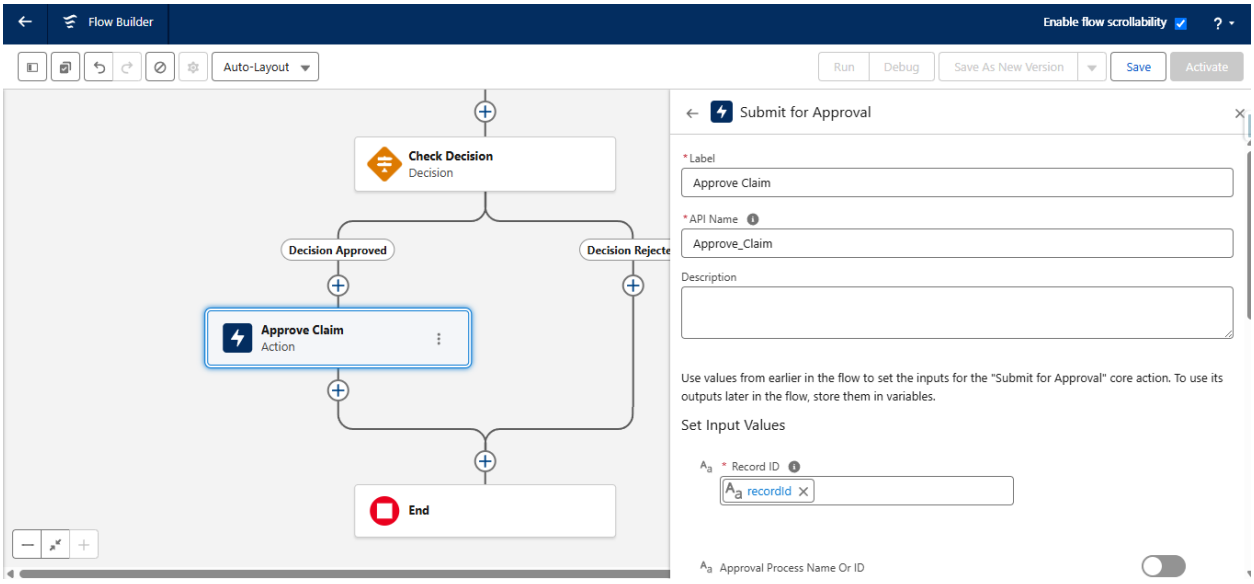


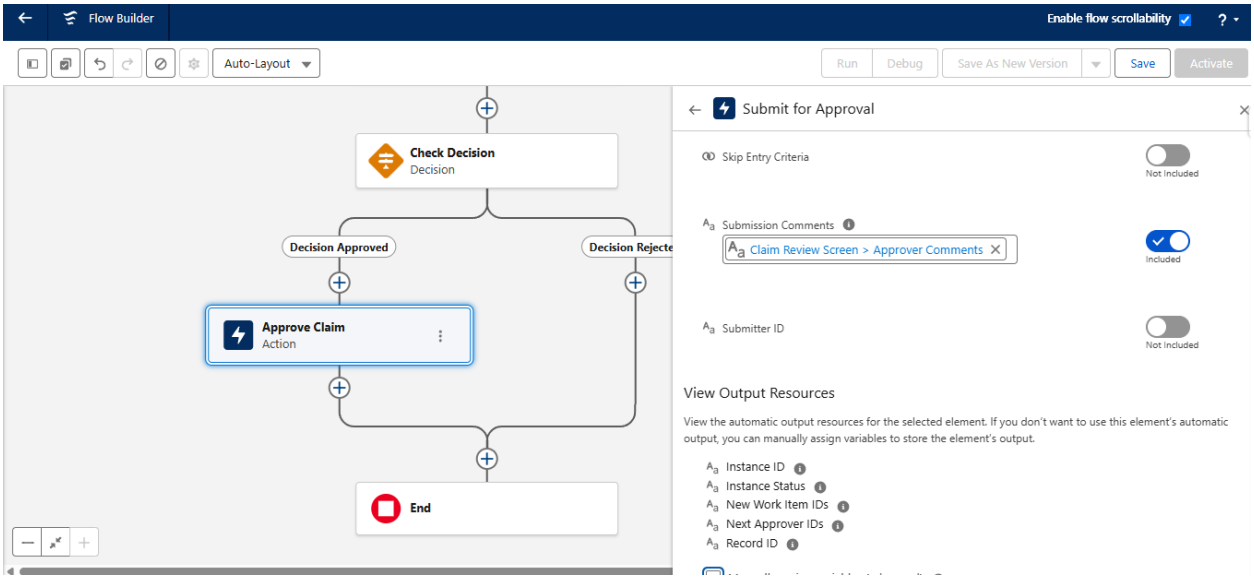
- Add Decision Element under the screen
- Label it as Check Decision
- Select Manually Select Conditions
- Set Outcome1:
    - Label it as Decision Approved
    - Select all conditions are met for conditional requirements
    - The condition includes:

    - Approve Decision From Claim Review Screen → Equals Approval Decision
- Set default Outcome:
    - Label it as Decision Rejected

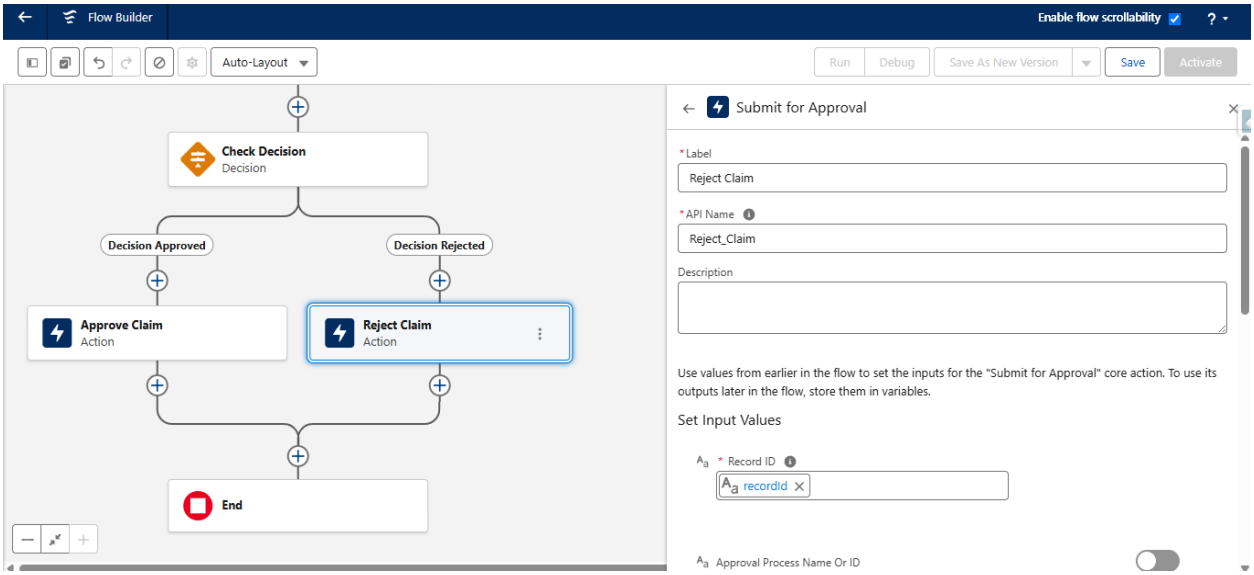


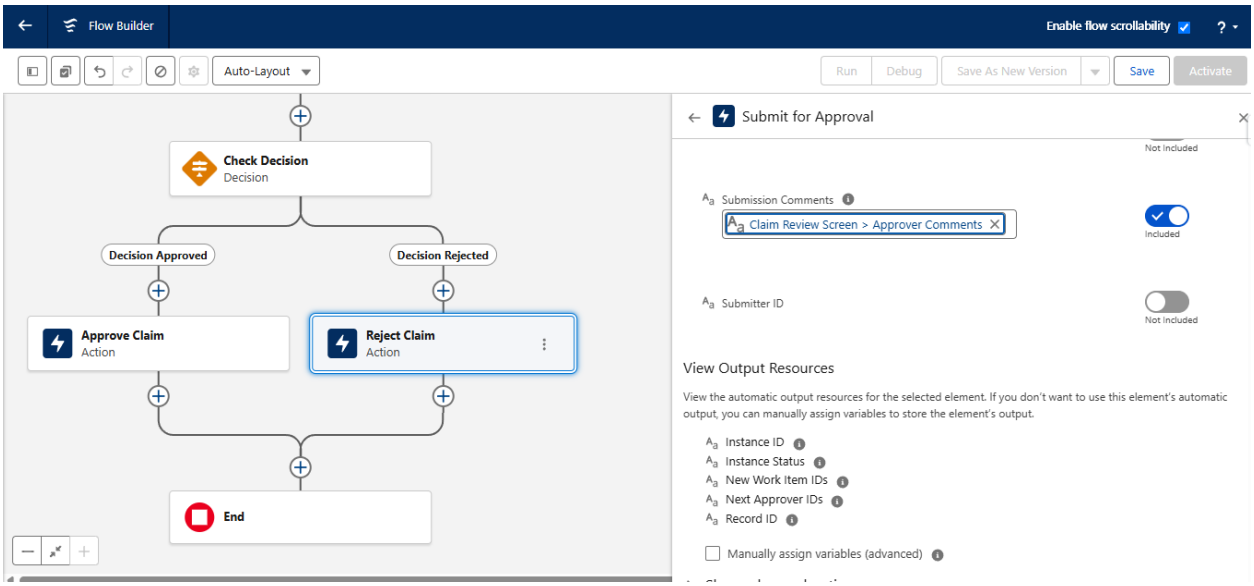
- Add Action Element Under Decision Approved Path
- Search for the submit for approval action and choose it
- Label it as Approve Claim
- Under Set Input Values
- Select variable recordID for Record ID
- Include the submission comments and Enter Approver Comments from Claim Review Screen





- Add Action Element Under Decision Rejected Path
- Search for the submit for approval action and choose it
- Label it as Reject Claim
- Under Set Input Values
- Select variable recordID for Record ID
- Include the submission comments and Enter Approver Comments from Claim Review Screen



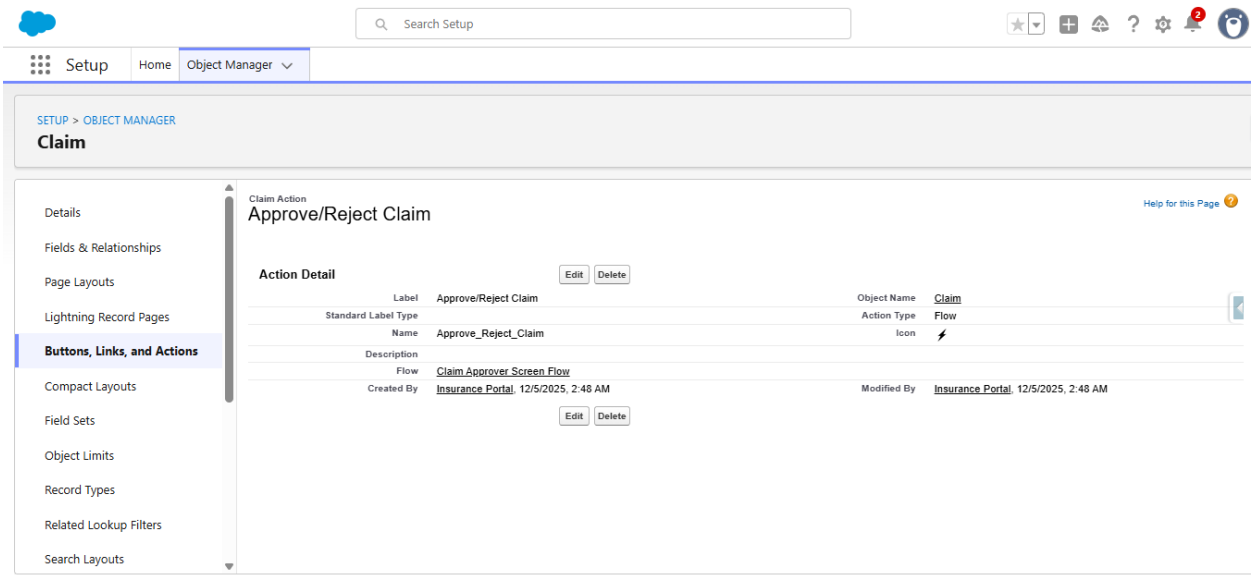


- Save and Activate the flow with Name Claim Approver Screen Flow

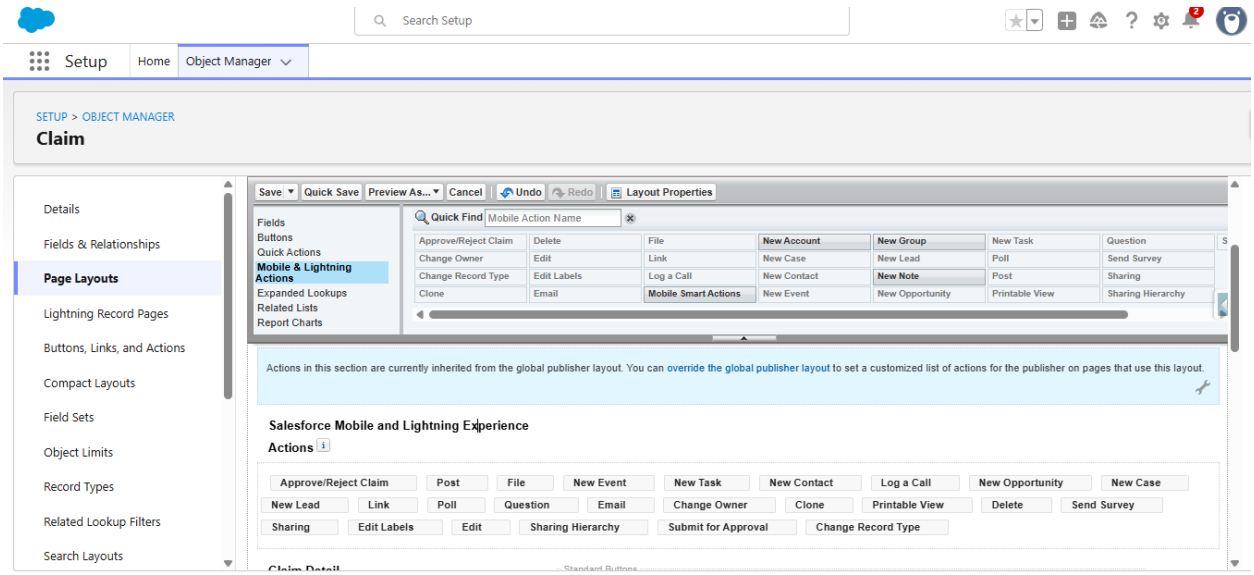
4.1 Create a Quick Action

- Go to setup, Click on Object Manager, Search for Claim Object and Select it

- In the left side panel, Click on Buttons, Links and Actions
- Click on Create New Action
- Select Action Type as Flow
- Select the flow that you have just created
- Label it as Approve/Reject Claim
- Name would be auto populated
- Click on Save



- Add the quick action to the claim page layout
- In the left panel, Click on page layouts and open the claim page layout
- Go to mobile & lightning actions, Find the Approve/Reject Claim and Drag it to the Salesforce Mobile and Lightning Experience Actions
- Click on Save



5. **Write an extensive Apex Test Class for the ClaimsAdjusterController with 95%+ code coverage. Include test methods for complex claim filtering and data retrieval logic.**
    - Click on the setup icon, Right click on Developer Console and click on open link in new tab.
    - On the top left corner, click on file, click on new, click on Apex class and give it a name ClaimsAdjusterControllerTest
    - Paste the following code:

```
@isTest

private class ClaimsAdjusterControllerTest {

    @testSetup
```

```
    static void setupTestData() {

        // Create a real active test user

        Profile p = [SELECT Id FROM Profile WHERE Name
= 'Standard User' LIMIT 1];

        User u = new User(

            FirstName = 'Test',

            LastName = 'User',

            Alias = 'tuser',

            Username = 'testuser1234343@example.com.test',

            Email = 'testuser123@example.com',

            TimeZoneSidKey = 'America/Los_Angeles',

            LocaleSidKey = 'en_US',

            EmailEncodingKey = 'UTF-8',

            LanguageLocaleKey = 'en_US',

            ProfileId = p.Id
```

```
        );

        insert u;



        // Query Policy__c RecordType

        Id policyRT =
Schema.SObjectType.Policy__c.getRecordTypeInfosByDe
veloperName()

            .get('Auto')

            .getRecordTypeId();

        Id claimRT =
Schema.SObjectType.Claim__c.getRecordTypeInfosByDe
veloperName()

            .get('Accident')

            .getRecordTypeId();



        // Create Contact owned by same user

        Contact con = new Contact(
```

```
            FirstName = 'John',
            LastName = 'Doe',
            OwnerId = u.Id
        );
        insert con;

        // Create Policy owned by same user
        Policy__c policy = new Policy__c(
            Customer__c = con.Id,
            RecordTypeId = policyRT,
            OwnerId = u.Id,
            VIN__c = '1A903284375893532'
        );
        insert policy;
```

```
        // Create Claim owned by same user
        Claim__c claim = new Claim__c(
            Policy__c = policy.Id,
            RecordTypeId = claimRT,
            Claim_Amount__c = 5000,
            Date_of_Loss__c = Date.today().addDays(-10),
            OwnerId = u.Id
        );
        insert claim;
        System.debug('Claim inserted: '+claim.Id);
    }
    @isTest
    static void testGetAssignedClaims() {
        User u = [SELECT Id FROM User WHERE Alias =
'tuser' LIMIT 1];
```

```
        System.runAs(u) {

            Test.startTest();

            List<ClaimsAdjusterController.ClaimWrapper>
results =

                ClaimsAdjusterController.getAssignedClaims();

            System.debug('Result is: '+results);

            Test.stopTest();

            //System.assert(results.size() > 0,

                //'Expected at least one wrapper returned');



            ClaimsAdjusterController.ClaimWrapper wrap =
results[0];



            System.assertNotEquals(null, wrap.claimId);

            System.assertNotEquals(null, wrap.claimNumber);

            System.assertEquals('New', wrap.status);
```

```
            System.assertEquals(5000, wrap.claimAmount);


System.assert(wrap.policyHolderName.contains('John'));

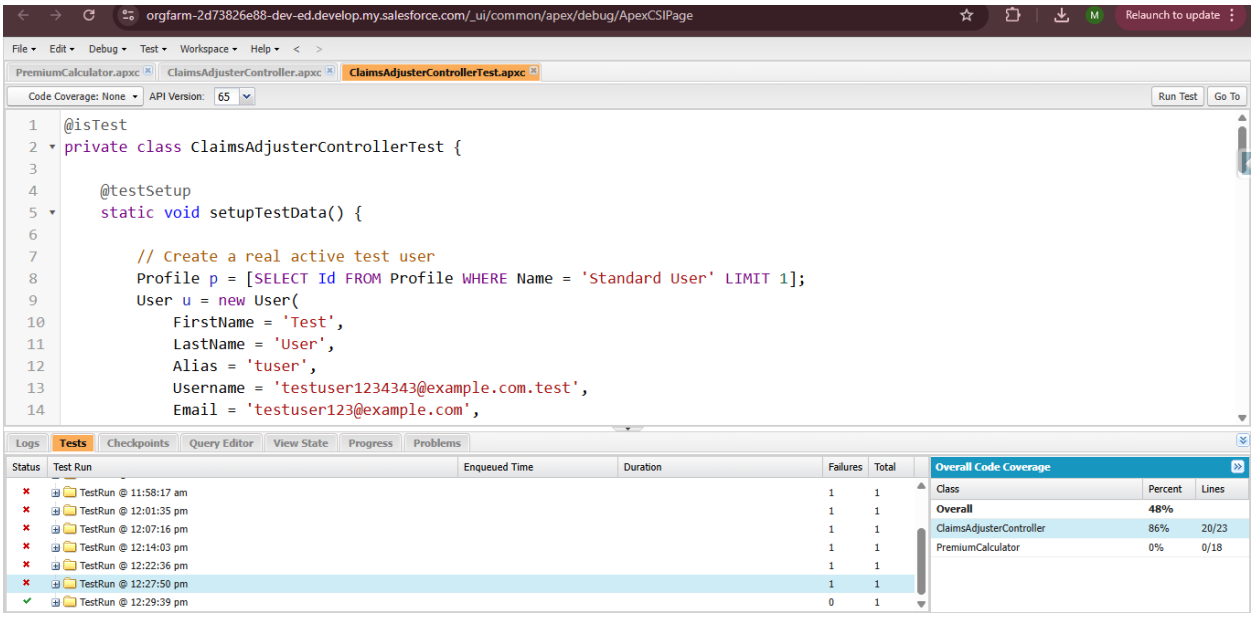
            System.assertNotEquals('Unknown',
wrap.policyType);

        }

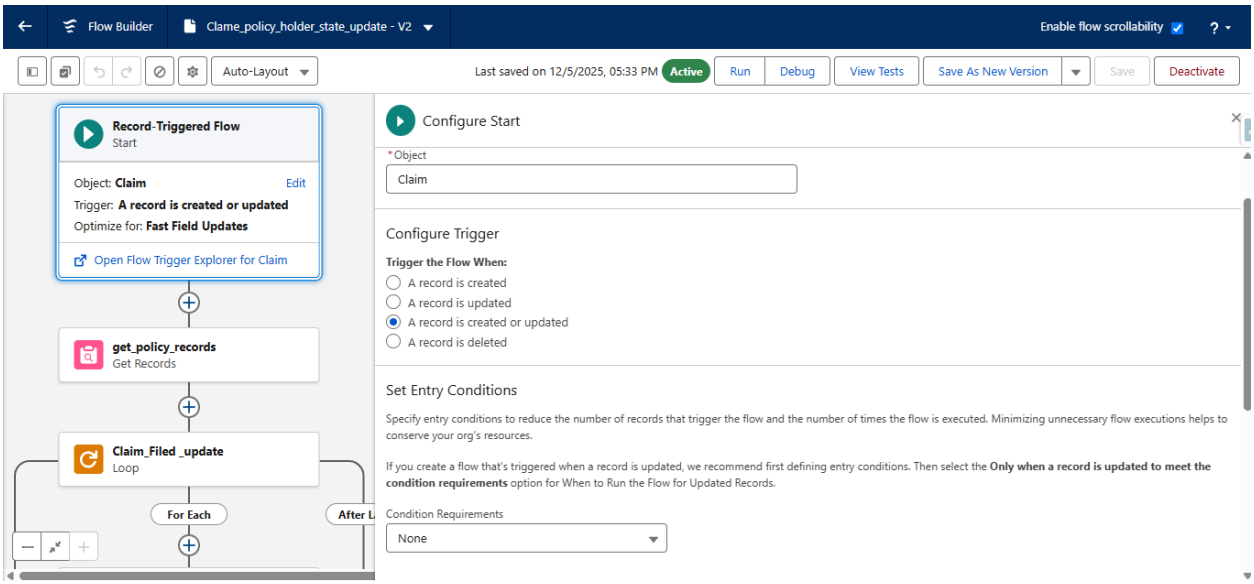
    }

}
```

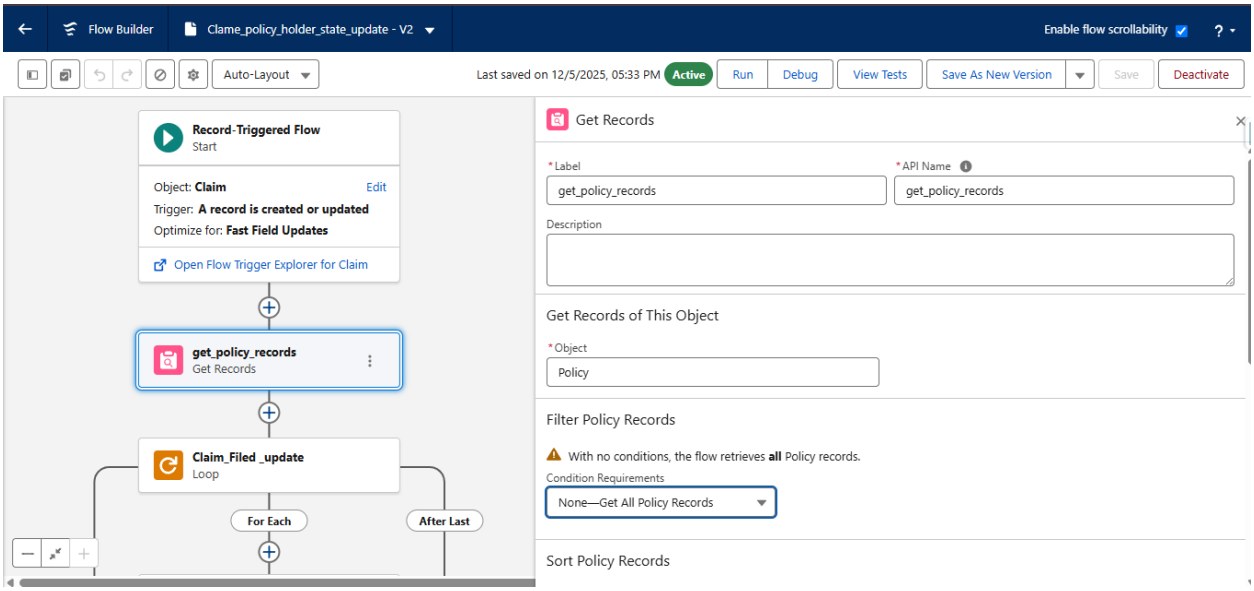
- Save the file using CTRL+S
- On the top right corner, you will find Run Test, Click on it to run the test class
- To check the test results, Down go for tests and you can see the how much percentage it covers your apex class

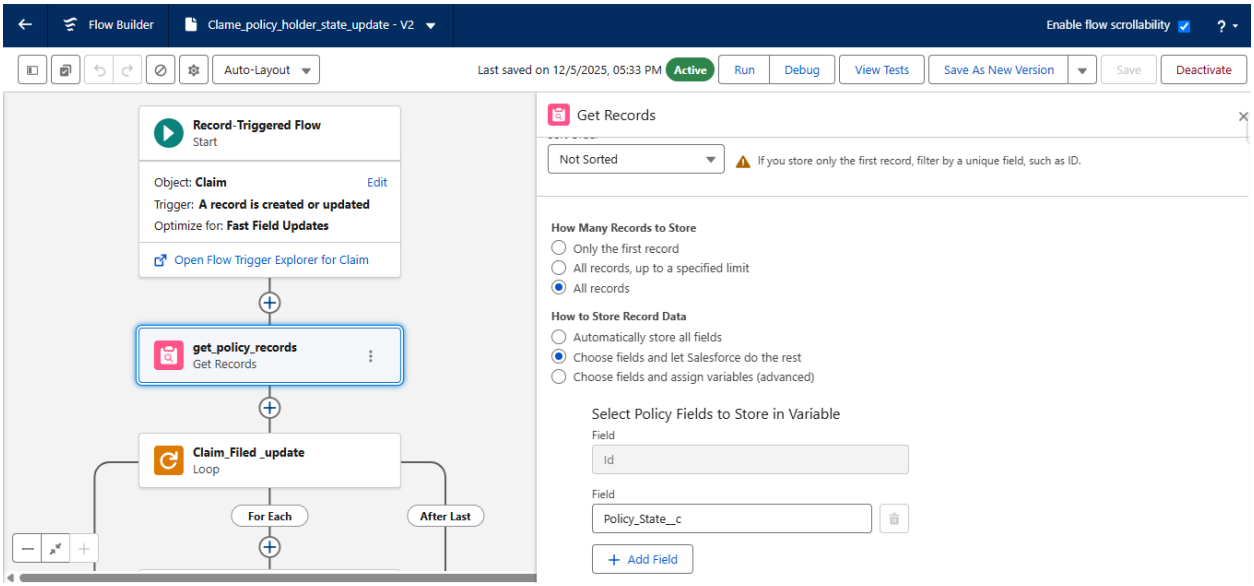


6. **Refine the Sharing Rules to ensure Adjusters only see claims in their assigned territory/state.**
    - Go to setup, Search for flows in the quick find box and click on flows
    - Click on New Flow
    - Select Record Triggered Flow
    - Select the object as claim
    - Configure trigger when a record is created or updated
    - Set entry conditions as none
    - Optimize the flow for Updates

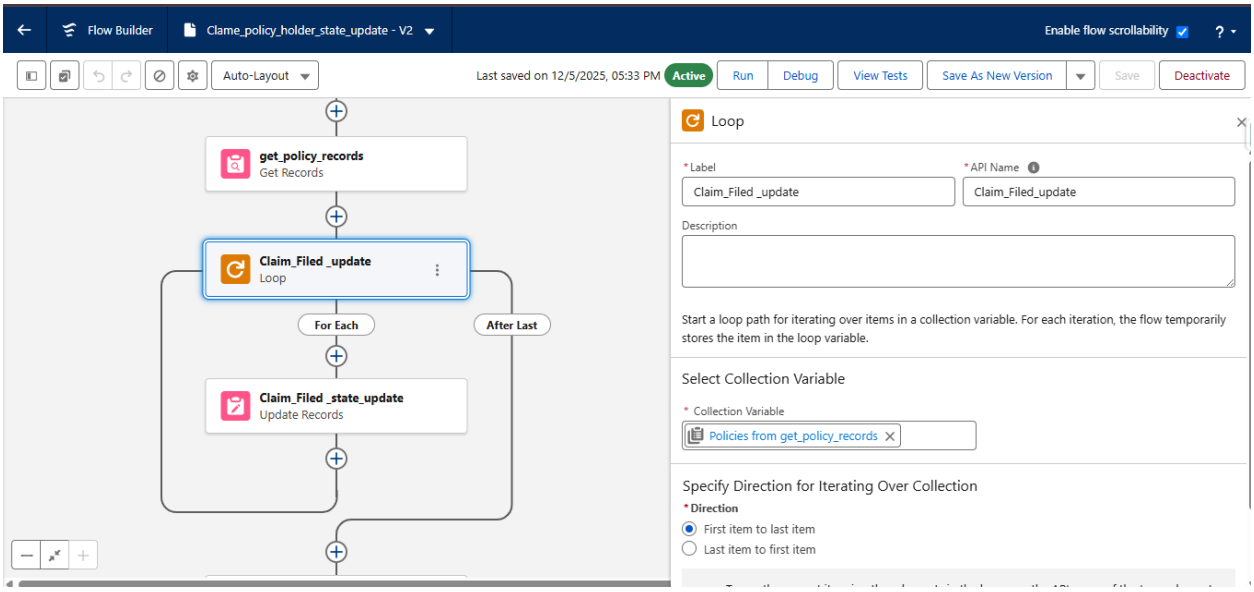


- Add get Records element into the flow canvas
- Label it as get_policy_records
- API name will be auto populated
- Set filter as None
- Store all the records
- Select Choose fields and let salesforce do the rest for How to store Record Data
- Under Select Policy fields to store fields in Variable, Select Policy State Field

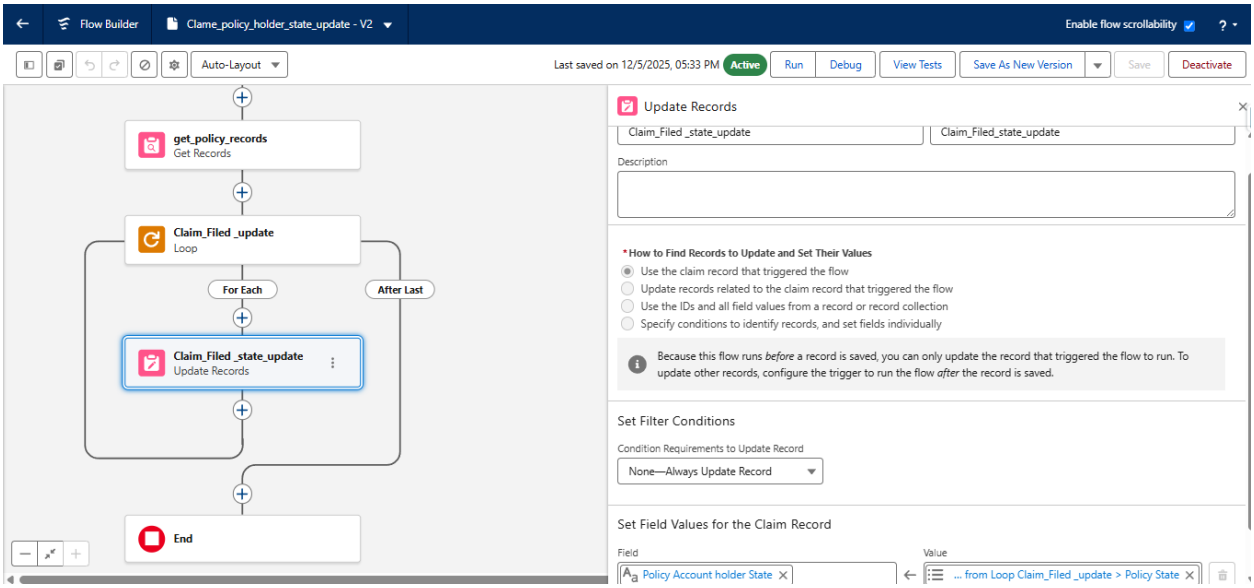




- Add the Loop Element into the canvas after the get Records element
- Label it as Claim_Filed _update
- API name will be auto populated
- Under Select Collection Variables, Select policies from Get Policies from Get Policy Records
- Under Specify direction for Iterating over collection, Select first item to last item



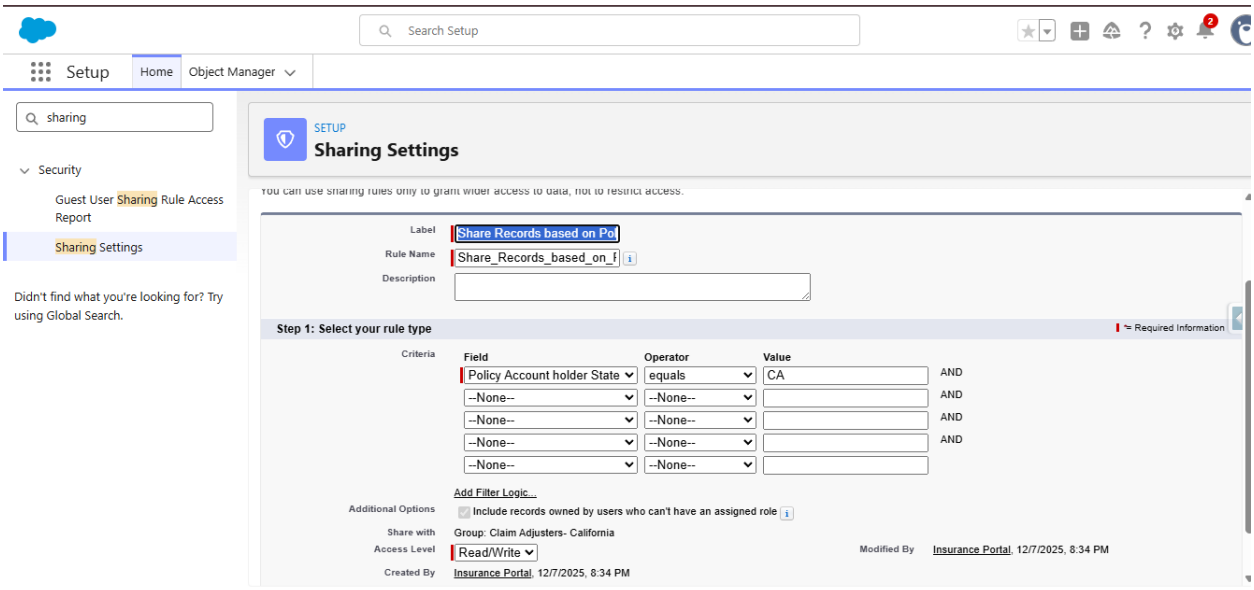
- Under For each, Add the update records element into the canvas
- Label it as Claim_Filed _state_update
- API name will be auto populated
- Set filter conditions to NONE
- Under Set field values for the claim record, Select policy state from current loop and set it to the Policy Account Holder State field



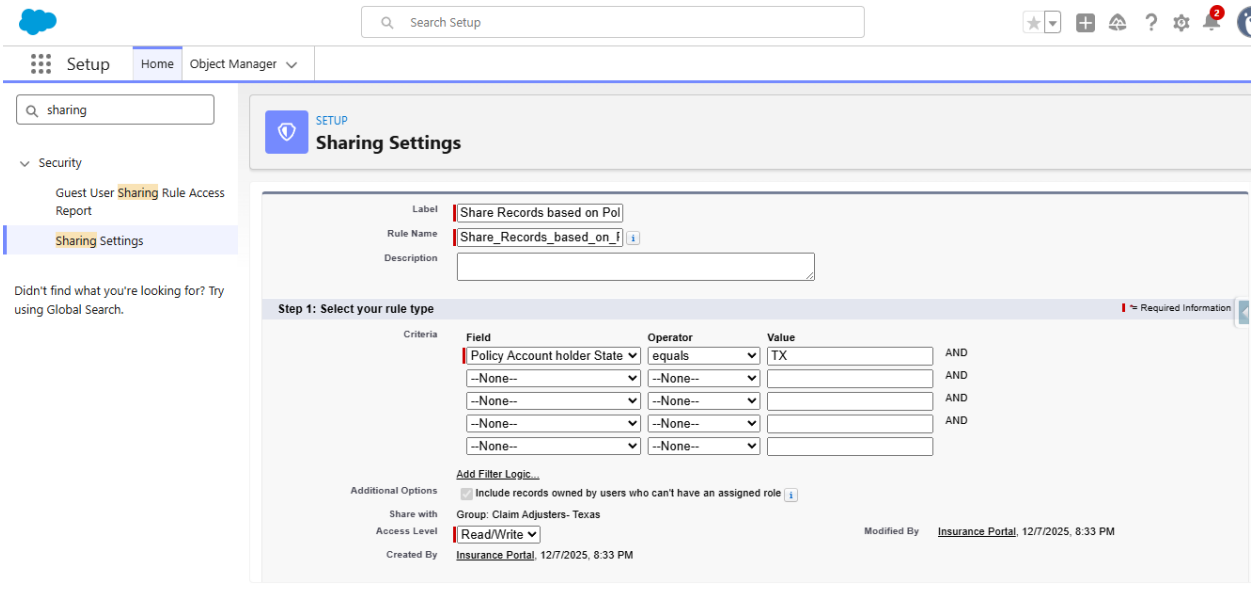
- Save and Activate the flow with name Claim Policy Holder State Update

6.1 Define Sharing Rules for Claim Object

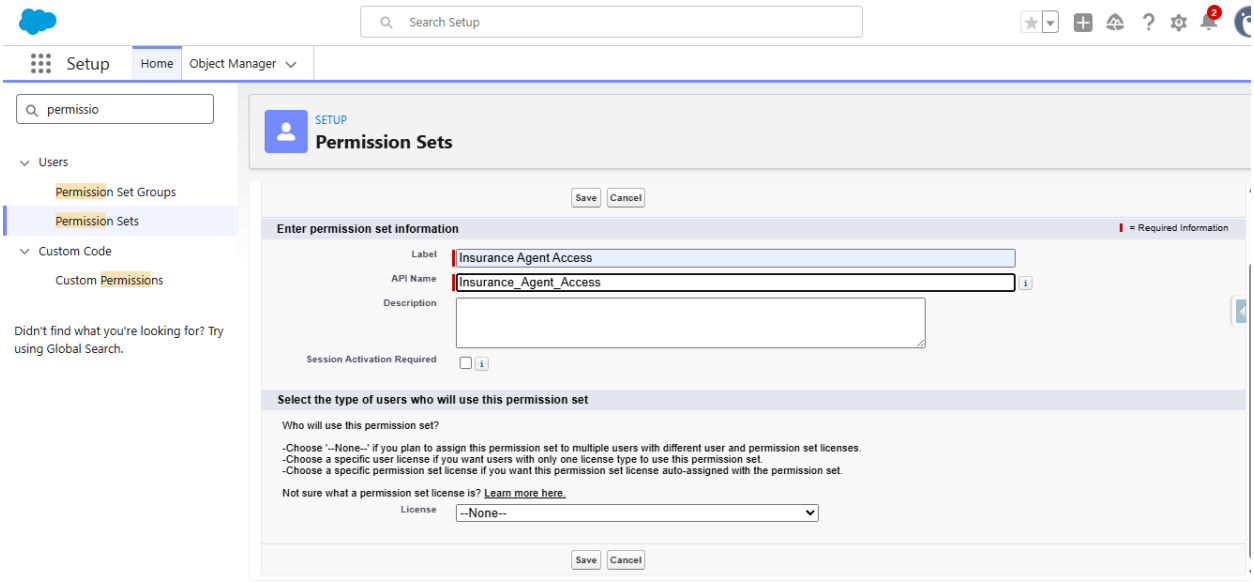
- Go to Set up, In the quick find box, Search for Sharing Settings
- Click on Sharing settings
- Under Manage Sharing Settings for, Select Claim Object
- Scroll down to find Claim Sharing Rules
- Click on new
- Label it as Share Records based on Policy State CA
- Rule Name will be Auto Populated
- Set the criteria to Policy Account Holder State Equals CA
- Give Access Level as Read/Write
- Click on Save



- Again Click on New
- Label it as Share Records based on Policy State
- Rule Name will be Auto Populated
- Set the criteria to Policy Account Holder State Equals TX
- Give Access Level as Read/Write
- Click on Save



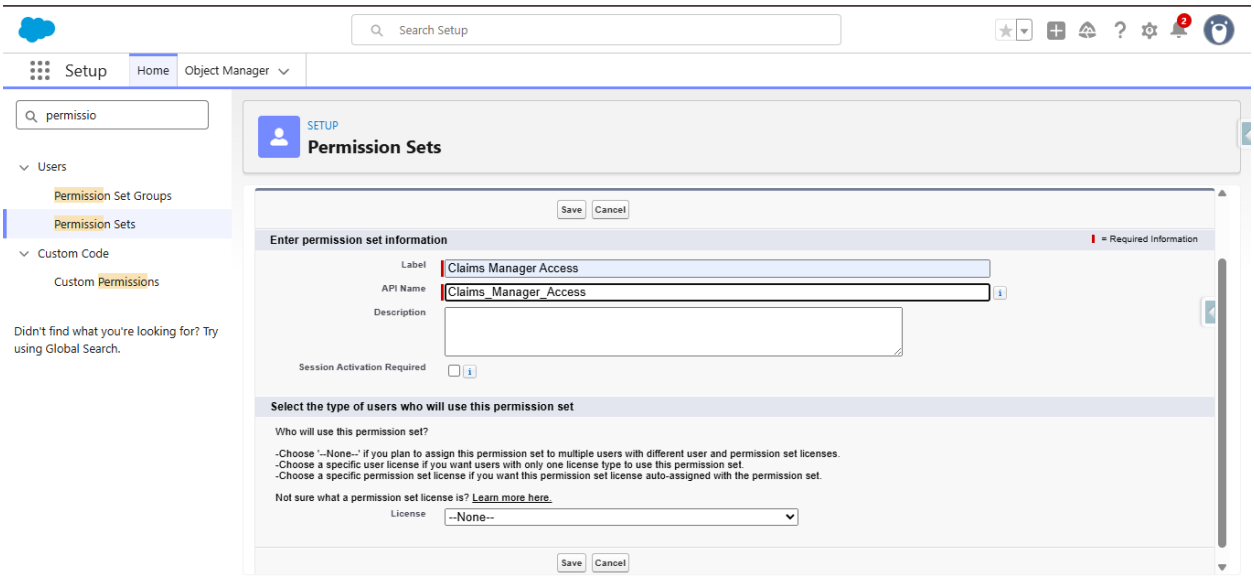
7. **Finalize all Permission Sets to differentiate access between Agents (Quoting only), Adjusters (Claim handling), and Managers (Reporting/Approvals).**
    - Go to setup, In the quick find box, Search for Permission Sets, Click on Permission Sets
    - Click on New
    - Give the Label as Insurance Agent Access
    - API Name will be Auto Populated



- Give the access for Objects
- Give Create, Read for Policy object
- Give Read, Edit to contact Object
- Give Read only to Claim object
- Make Flow User Enabled under System permissions
- Make PremiumCalculator Apex class enabled

7.1 Create Claims Manager Access Permission set

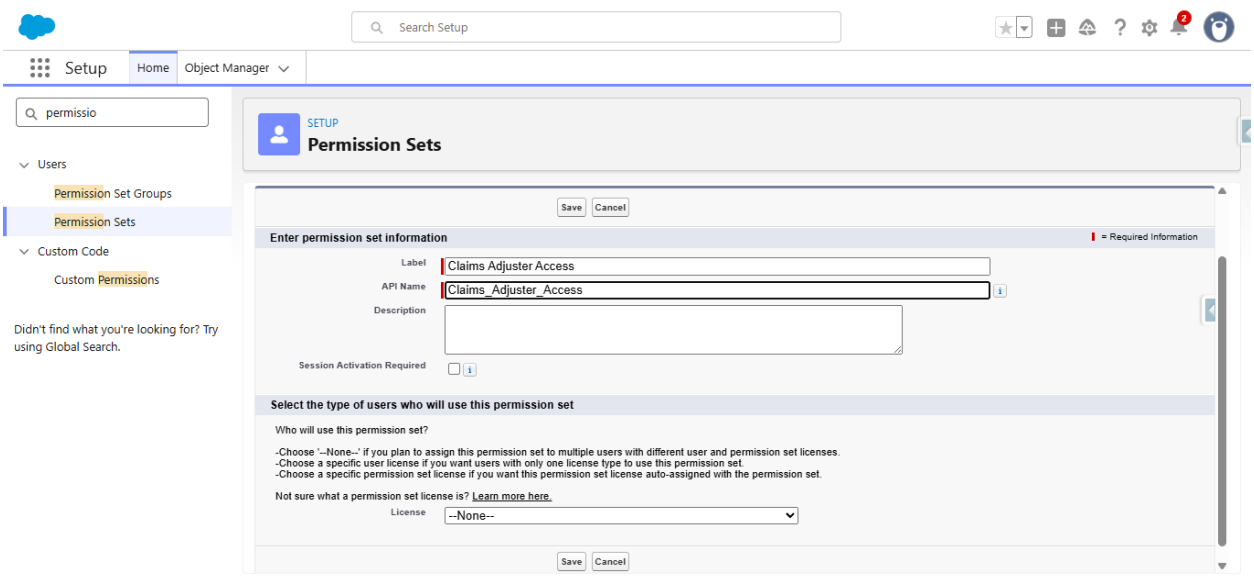
- Click on New
- Give the Label as Claims Manager Access
- API Name will be Auto Populated



- Give the access to objects
- Give Read Only for Policy object
- Give Read, Edit to contact Object
- Give Read, Edit, Delete to Claim object
- Make Run Reports Enabled under System permissions
- Make Manage Approvals enabled under system permissions

7.2 Create Claims Adjuster Access Permission Set

- Click on New
- Give the Label as Claims Adjuster Access
- API Name will be Auto Populated



- Give the access to objects
- Give Read Only for Policy object
- Give Read, Edit to Claim object
- Make ClaimsAdjusterController Enabled under Apex Classes